

第3部分 竞赛篇

第8章 高效算法设计

学习目标

- ☑ 理解“基本操作”、渐进时间复杂度的概念和大O记号的含义
- ☑ 掌握“最大连续和”问题的各种算法及其时间复杂度分析
- ☑ 正确认识算法分析的优点和局限性，能正确使用分析结果
- ☑ 掌握归并排序和逆序对统计的分治算法
- ☑ 理解快速排序和快速选择算法
- ☑ 熟练掌握二分查找算法，包括找上下界的算法
- ☑ 能用递归的方式思考和求解问题
- ☑ 熟练掌握用二分法求解非线性方程的方法
- ☑ 熟练掌握用二分法把优化问题转化为判定问题的方法
- ☑ 熟悉能用贪心法求解的各类经典问题
- ☑ 掌握本章中介绍的各种算法设计思路与方法

尽管直观、适用范围广，但枚举、回溯等暴力方法常常无法走出“低效”的阴影。这并不难理解：越是通用的算法，越不能深入挖掘问题的特殊性。本章介绍一些经典问题的高效算法。由于是“量身定制”的，这些算法从概念、思路到程序实现都是千差万别的。从某种意义上说，从本章开始，读者才刚刚开始接触“严肃”的算法设计理论。

8.1 算法分析初步

编程者都希望自己的算法高效，但算法在写成程序之前是运行不了的。难道每设计出来一个算法都必须写出程序来才知道快不快吗？答案是否定的。本节介绍算法分析的基本概念和方法，力求在编程之前尽量准确地估计程序的时空开销，并作出决策——例如，如果算法又复杂速度又慢，就不要急着写出来了。

8.1.1 渐进时间复杂度

最大连续和问题。给出一个长度为 n 的序列 $A_1, A_2, \dots, A_n$ ，求最大连续和。换句话说，要求找到 $1 \leq i \leq j \leq n$ ，使得 $A_i + A_{i+1} + \dots + A_j$ 尽量大。

【分析】

使用枚举，得出如下程序：

程序 8-1 最大连续和 (1)

```
tot = 0;
best = A[1]; //初始最大值
for(int i = 1; i <= n; i++)
    for(int j = i; j <= n; j++){ //检查连续子序列 A[i], ..., A[j]
        int sum = 0;
        for(int k = i; k <= j; k++) { sum += A[k]; tot++; } //累加元素和
        if(sum > best) best = sum; //更新最大值
    }
```

注意 best 的初值是 A[1]，这是最保险的做法——不要写 best=0（想一想，为什么）。当 $n=1000$ 时，输出 tot=167167000，这是加法运算的次数。当 $n=50$ 时，输出 22100。

为什么要计算 tot 呢？因为它与机器的运行速度无关。不同机器的速度不一样，运行时间也会有所差异，但 tot 值一定相同。换句话说，它去掉了机器相关的因素，只衡量算法的“工作量”大小——具体来说，是“加法”操作的次数。

提示 8-1：统计程序中“基本操作”的数量，可以排除机器速度的影响，衡量算法本身的优劣程度。

在本题中，将“加法操作”作为基本操作，类似地也可以把其他四则运算、比较运算作为基本操作。一般并不会严格定义基本操作的类型，而是根据不同情况灵活处理。

刚才实验得出 tot 值的，其实它也可以用数学方法直接推导出。设输入规模为 n 时加法操作的次数为 $T(n)$ ，则：

$$T(n) = \sum_{i=1}^n \sum_{j=i}^n j - i + 1 = \sum_{i=1}^n \frac{(n-i+1)(n-i+2)}{2} = \frac{n(n+1)(n+2)}{6}$$

上面的公式是关于 n 的三次多项式，意味着当 n 很大时，平方项和一次项对整个多项式值的影响不大。可以用一个记号来表示： $T(n) = \Theta(n^3)$ ，或者说 $T(n)$ 和 n^3 同阶。

同阶是什么意思呢？简单地说，就是“增长情况相同”。前面说过， n 很大时，只有立方项起到决定作用，而立方项的系数对“增长”是不起作用的—— n 扩大两倍时， n^3 和 $100n^3$ 都扩大 8 倍。这样一来，可以只保留“最大项”，并忽略其系数，得到的简单式子称为算法的渐进时间复杂度（asymptotic time complexity）。

提示 8-2：基本操作的数量往往可以写成关于“输入规模”的表达式，保留最大项并忽略系数后的简单表达式称为算法的渐进时间复杂度，用于衡量算法中基本操作数随规模的增长情况。

读者可以做个实验，看看 n 扩大两倍时运行时间是否近似扩大 8 倍。注意这里的“8 倍”是近似的，因为在 $T(n)$ 的表达式中，二次项、一次项和常数项都被忽略掉了；程序中的其他运算，如 if(sum > best) 中的比较运算，甚至改变循环变量所需的“自增”都没有考虑在内。

尽管如此，算法分析的效果还是比较精确的，因为抓住了主要矛盾——执行得最多的运算是加法。

提示 8-3：渐进时间复杂度忽略了很多因素，因而分析结果只能作为参考，并不是精确的。尽管如此，如果成功抓住了最主要的运算量所在，算法分析的结果常常十分有用。

8.1.2 上界分析

对于上面的方法，读者可能会有疑问：难道每次都要作一番复杂的数学推导才能得到渐进时间复杂度吗？当然不必。

下面是另外一种推导方法：算法包含 3 重循环，内层最坏情况下需要循环 n 次，中层循环最坏情况下也需要 n 次，外层循环最坏情况下仍然需要 n 次，因此总运算次数不超过 n^3 。这里采用了“上界分析”，假定所有最坏情况同时取到，尽管这是不可能的。不难预料，这样的分析和实际情况肯定会有一定偏差——在 $T(n)$ 的表达式中， n^3 的系数是 $1/6$ ，小于 n^3 ，但数量级是正确的——仍然可以得到“ n 扩大两倍时，运行时间近似扩大 8 倍”的结论。上界也有记号： $T(n)=O(n^3)$ 。

提示 8-4：在算法设计中，常常不进行精确分析，而是假定各种最坏情况同时取到，得到上界。在很多情况下，这个上界和实际情况同阶（称为“紧”的上界），但也有可能会因为分析方法不够好，得到“松”的上界。

松的上界也是正确的上界，但可能让人过高估计程序运行的实际时间（从而不敢编写程序），而即使上界是紧的，过大（如 100）或过小（如 $1/100$ ）的最高项系数同样可能引起错误的估计。换句话说，算法分析不是万能，要谨慎对待分析结果。如果预感到上界不紧、系数过大或者过小，最好还是要编程实践。

下面试着优化一下这个算法。设 $S_i=A_1+A_2+\cdots+A_i$ ，则 $A_i+A_{i+1}+\cdots+A_j=S_j-S_{i-1}$ 。该式子的用途相当广泛，其直观含义是“连续子序列之和等于两个前缀和之差”。有了这个结论，最内层的循环就可以省略了。

程序 8-2 最大连续和（2）

```
S[0] = 0;
for(int i = 1; i <= n; i++) S[i] = S[i-1] + A[i];           //递推前缀和 S
for(int i = 1; i <= n; i++)
    for(int j = i; j <= n; j++) best = max(best, S[j]-S[i-1]); //更新最大值
```

注意上面的程序用到了递推的思想：从小到大依次计算 $S[1], S[2], S[3], \dots$ ，每个只需要在前一个的基础上加上一个元素。换句话说，“计算 S ”这个步骤的时间复杂度为 $O(n)$ 。接下来是一个二重循环，用类似的方法可以分析出：

$$T(n) = \sum_{i=1}^n n-i+1 = \frac{n(n+1)}{2}$$

代入可得 $T(1000)=500500$ ，和运行结果一致。同样地，用上界分析可以更快地得到结

论：内层循环最坏情况下要执行 n 次，外层也是，因此时间复杂度为 $O(n^2)$ 。

8.1.3 分治法

本节使用分治法来解决这个问题。分治算法一般分为如下 3 个步骤。

划分问题：把问题的实例划分成子问题。

递归求解：递归解决子问题。

合并问题：合并子问题的解得到原问题的解。

在本例中，“划分”就是把序列分成元素个数尽量相等的两半；“递归求解”就是分别求出完全位于左半或者完全位于右半的最佳序列；“合并”就是求出起点位于左半、终点位于右半的最大连续和序列，并和子问题的最优解比较。

前两部分没有什么特别之处，关键在于“合并”步骤。既然起点位于左半，终点位于右半，则可以人为地把这样的序列分成两部分，然后独立求解：先寻找最佳起点，然后再寻找最佳终点。

程序 8-3 最大连续和 (3) (如图 8-1 所示)

```
int maxsum(int* A, int x, int y){ //返回数组在左闭右开区间[x,y)中的最大连续和
    int v, L, R, maxs;
    if(y - x == 1) return A[x]; //只有一个元素，直接返回
    int m = x + (y-x)/2; //分治第一步：划分成[x, m)和[m, y)
    int maxs = max(maxsum(A, x, m), maxsum(A, m, y)); //分治第二步：递归求解
    int v, L, R;
    v = 0; L = A[m-1]; //分治第三步：合并(1)——从分界点开始往左的最大连续和L
    for(int i = m-1; i >= x; i--) L = max(L, v += A[i]);
    v = 0; R = A[m]; //分治第三步：合并(2)——从分界点开始往右的最大连续和R
    for(int i = m; i < y; i++) R = max(R, v += A[i]);
    return max(maxs, L+R); //把子问题的解与L和R比较
}
```

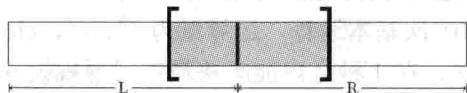


图 8-1 最大连续和的分治算法

上面的代码用到了“赋值运算本身具有返回值”的特点，在一定程度上简化了代码，但不会牺牲可读性。

在上面的程序中， L 和 R 分别为从分界线往左、往右能达到的最大连续和。对于 $n=1000$ ， tot 值仅为 9976，在前面的 $O(n^2)$ 算法基础上又有大幅度改进。

是否可以像前面那样，得到 tot 的数学表达式呢？注意求和技巧已经不再适用，需要用递归的思路进行分析：设序列长度为 n 时的 tot 值为 $T(n)$ ，则 $T(n)=2T(n/2)+n$ ， $T(1)=1$ 。其中 $2T(n/2)$ 是两次长度为 $n/2$ 的递归调用，而最后的 n 是合并的时间（整个序列恰好扫描一遍）。注意这个方程是近似的，因为当 n 为奇数时两次递归的序列长度分别为 $(n-1)/2$ 和

$(n+1)/2$ ，而不是 $n/2$ 。幸运的是，这样的近似对于最终结果影响很小，在分析算法时总是可以忽略它。

提示 8-5：在算法分析中，往往可以忽略“除法结果是否为整数”，而直接按照实数除法分析。这样的近似对最终结果影响很小，一般不会改变渐进时间复杂度。

解刚才的方程，可以得到 $T(n) = \Theta(n \log n)$ 。由于 $n \log n$ 增长很慢，当 n 扩大两倍时，运行时间的扩大倍数只是略大于 2。现在不必懂得解方程的方法，可以把它作为一个重要结论记下来（建议有兴趣的读者试着借助于解答树来证明这个结论，它并不复杂）。

提示 8-6：递归方程 $T(n) = 2T(n/2) + \Theta(n)$ ， $T(1)=1$ 的解为 $T(n) = \Theta(n \log n)$ 。

在结束对分治算法的讨论之前，有必要再谈谈上述程序中的两个细节。首先是范围表示。上面的程序用左闭右开区间来表示一个范围，好处是在处理“数组分割”时比较自然：区间 $[x, y)$ 被分成的是 $[x, m)$ 和 $[m, y)$ ，不需要在任何地方加减 1。另外，空区间表示为 $[x, x)$ ，比 $[x, x-1]$ 顺眼多了。

另一个细节是“分成元素个数尽量相等的两半”时分界点的计算。在数学上，分界点应当是 x 和 y 的平均数 $m=(x+y)/2$ ，此处用的却是 $x+(y-x)/2$ 。在数学上二者相等，但在计算机中却有差别。不知读者是否注意到，运算符“/”的“取整”是朝零方向（towards zero）的取整，而不是向下取整。换句话说， $5/2$ 的值是 2，而 $-5/2$ 的值是 -2。为了方便分析，此处用 $x+(y-x)/2$ 来确保分界点总是靠近区间起点。这在本题中并不是必要的，但在后面要介绍的二分查找中，却是相当重要的技巧。

8.1.4 正确对待算法分析结果

对于“最大连续和”问题，本书先后介绍了时间复杂度为 $O(n^3)$ 、 $O(n^2)$ 、 $O(n \log n)$ 的算法，每个新算法较前一个来说，都是重大的改进。尽管分治法看上去很巧妙，但并不是最高效的。把 $O(n^2)$ 算法稍作修改，便可以得到一个 $O(n)$ 算法：当 j 确定时，“ $S[j]-S[i-1]$ 最大”相当于“ $S[i-1]$ 最小”，因此只需要扫描一次数组，维护“目前遇到过的最小 S ”即可。

假设机器速度是每秒 10^8 次基本运算，运算量为 n^3 、 n^2 、 $n \log_2 n$ 、 n 、 2^n （如子集枚举）和 $n!$ （如排列枚举）的算法，在 1 秒之内能解决最大问题规模 n ，如表 8-1 所示。

表 8-1 运算量随着规模的变化

运算量	$n!$	2^n	n^3	n^2	$n \log_2 n$	n
最大规模	11	26	464	10000	4.5×10^6	100000000
速度扩大两倍后	11	27	584	14142	8.6×10^6	200000000

表 8-1 还给出了机器速度扩大两倍后，算法所能解决规模的对比。可以看出， $n!$ 和 2^n 不仅能解决的问题规模非常小，而且增长缓慢；最快的 $n \log_2 n$ 和 n 算法不仅解决问题的规模大，而且增长快。渐进时间复杂为多项式的算法称为多项式时间算法（polynomial-time algorithm），也称有效算法；而 $n!$ 或者 2^n 这样的低效的算法称为指数时间算法（exponential-time algorithm）。

不过需要注意的是，上界分析的结果在趋势上能反映算法的效率，但有两个不精确性：一是公式本身的不精确性。例如，“非主流”基本操作的影响、隐藏在大 O 记号后的低次项和最高项系数；二是对程序实现细节与计算机硬件的依赖性，例如，对复杂表达式的优化计算、把内存访问方式设计得更加“cache 友好”等。在不少情况下，算法实际能解决的问题规模与表 8-1 所示有着较大差异。

尽管如此，表 8-1 还是有一定借鉴意义的。考虑到目前主流机器的执行速度，多数算法竞赛题目所选取的数据规模基本符合此表。例如，一个指明 $n \leq 8$ 的题目，可能 $n!$ 的算法已经足够， $n \leq 20$ 的题目需要用到 2^n 的算法，而 $n \leq 300$ 的题目可能必须用至少 n^3 的多项式时间算法了。

8.2 再谈排序与检索

假设有 n 个整数，希望把它们按照从小到大的顺序排列，应该怎样做呢？也许你会说：调用 STL 中的 `sort` 或者 `stable_sort` 即可。可是读者们有没有想过：这些现成的排序函数是怎样工作的呢？

8.2.1 归并排序

第一种高效排序算法是归并排序。按照分治三步法，对归并排序算法介绍如下。

划分问题：把序列分成元素个数尽量相等的两半。

递归求解：把两半元素分别排序。

合并问题：把两个有序表合并成一个。

前两部分是很容易完成的，关键在于如何把两个有序表合成一个。图 8-2 演示了一个合并的过程。每次只需要把两个序列的最小元素加以比较，删除其中的较小元素并加入合并后的新表即可。由于需要一个新表来存放结果，所以附加空间为 n 。

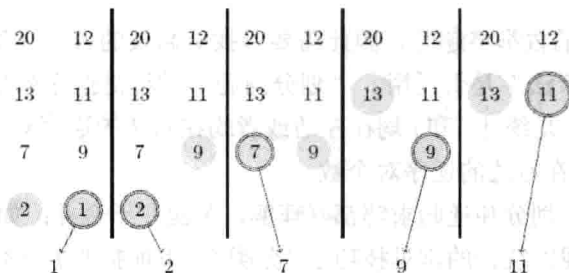


图 8-2 合并过程：时间是线性的，需要线性的辅助空间

这个过程极为重要，希望读者仔细体会。代码如下：

程序 8-4 归并排序（从小到大）

```
void merge_sort(int* A, int x, int y, int* T){
    if(y-x > 1){
```

```

int m = x + (y-x)/2; //划分
int p = x, q = m, i = x;
merge_sort(A, x, m, T); //递归求解
merge_sort(A, m, y, T); //递归求解
while(p < m || q < y){
    if(q >= y || (p < m && A[p] <= A[q])) T[i++] = A[p++];
    //从左半数组复制到临时空间
    else T[i++] = A[q++]; //从右半数组复制到临时空间
}
for(i = x; i < y; i++) A[i] = T[i]; //从辅助空间复制回 A 数组
}
}

```

代码中的两个条件是关键。首先，只要有一个序列非空，就要继续合并（ $\text{while}(p < m \mid q < y)$ ），因此在比较时不能直接比较 $A[p]$ 和 $A[q]$ ，因为可能其中一个序列为空，从而 $A[p]$ 或者 $A[q]$ 代表的是一个实际不存在的元素。正确的方式是：

- 如果第二个序列为空（此时第一个序列一定非空），复制 $A[p]$ 。
- 否则（第二个序列非空），当且仅当第一个序列也非空，且 $A[p] \leq A[q]$ 时，才复制 $A[p]$ 。

上面的代码巧妙地利用短路运算符“ $\mid$ ”把两个条件连接在了一起：如果条件 1 满足，就不会计算条件 2；如果条件 1 不满足，就一定会计算条件 2。这样的技巧很实用，请读者细心体会。另外，读者如果仍然不太习惯 $T[i++] = A[p++]$ 这种“复制后移动下标”的方式，是时候把它们弄懂、弄熟了。

不难看出，归并排序的时间复杂度和最大连续和的分治算法一样，都是 $O(n \log n)$ 的。

逆序对问题。给一系列数 $a_1, a_2, \dots, a_n$ ，求它的逆序对数，即有多少个有序对 (i, j) ，使得 $i < j$ 但 $a_i > a_j$ 。 n 可以高达 10^6 。

【分析】

n 这么大， $O(n^2)$ 的枚举将超时，因此需要寻找更高效的方法。受到归并排序的启发，下面来试试“分治三步法”是否适用。“划分问题”过程是把序列分成元素个数尽量相等的两半；“递归求解”是统计 i 和 j 均在左边或者均在右边的逆序对个数；“合并问题”则是统计 i 在左边，但 j 在右边的逆序对个数。

和归并排序一样，划分和递归求解都好理解，关键在于合并：如何求出 i 在左边，而 j 在右边的逆序对数目呢？统计的常见技巧是“分类”。下面按照 j 的不同把这些“跨越两边”的逆序对进行分类：只要对于右边的每个 j ，统计左边比它大的元素个数 $f(j)$ ，则所有 $f(j)$ 之和便是答案。

幸运的是，归并排序可以“顺便”完成 $f(j)$ 的计算：由于合并操作是从小到大进行的，当右边的 $A[j]$ 复制到 T 中时，左边还没来得及复制到 T 的那些数就是左边所有比 $A[j]$ 大的数。此时在累加器中加上左边元素个数 $m-p$ 即可（左边所剩的元素在区间 $[p, m]$ 中，因此元素个数为 $m-p$ ）。换句话说，在代码上的唯一修改就是把“ $\text{else } T[i++] = A[q++]$ ；”改成“ else

{ T[i++] = A[q++]; cnt += m-p; }”。当然，在调用之前应给 cnt 清零。

提示 8-7：归并排序的时间复杂度为 $O(n\log n)$ 。对该算法稍加修改，可以统计序列中的逆序对的个数，时间复杂度不变。

8.2.2 快速排序

快速排序是最快的通用内部排序算法。它由 Hoare 于 1962 年提出，相对归并排序来说不仅速度更快，并且不需辅助空间（还记得那个 T 数组吗）。按照分治三步法，将快速排序算法作如下介绍。

划分问题：把数组的各个元素重排后分成左右两部分，使得左边的任意元素都小于或等于右边的任意元素。

递归求解：把左右两部分分别排序。

合并问题：不用合并，因为此时数组已经完全有序。

读者也许会觉得这样的描述太过笼统，但事实上，快速排序本来就不是只有一种实现方法。“划分过程”有多个不同的版本，导致快速排序也有不同版本。读者很容易在互联网上找到各种快速排序的版本，这里不再给出代码。

快速选择问题。输入 n 个整数和一个正整数 k ($1 \leq k \leq n$)，输出这些整数从小到大排序后的第 k 个（例如， $k=1$ 就是最小值）。 $n \leq 10^7$ 。

【分析】

选择第 k 大的数，最容易想到的方法是先排序，然后直接输出下标为 $k-1$ 的元素（别忘了 C 语言中数组下标从 0 开始），但 10^7 的规模即使对于 $O(n\log n)$ 的算法来说较大。有没有更快的方法呢？

答案是肯定的。假设在快速排序的“划分”结束后，数组 $A[p \cdots r]$ 被分成了 $A[p \cdots q]$ 和 $A[q+1 \cdots r]$ ，则可以根据左边的元素个数 $q-p+1$ 和 k 的大小关系只在左边或者右边递归求解。可以证明，在期望意义下，程序的时间复杂度为 $O(n)$ 。

提示 8-8：快速排序的时间复杂度为：最坏情况 $O(n^2)$ ，平均情况 $O(n\log n)$ ，但实践中几乎不可能达到最坏情况，效率非常高。根据快速排序思想，可以在平均 $O(n)$ 时间内选出数组中第 k 大的元素。

8.2.3 二分查找

排序的重要意义之一，就是为检索带来方便。试想有 10^6 个整数，希望确认其中是否包含 12345，最容易想到的方法就是把它们放到数组 A 中，然后依次检查这些整数是否等于 12345。这样的方式对于“单次询问”来说运行得很好，但如果需要找 10000 个数，就需要把整个数组 A 遍历 10000 次。而如果先将数组 A 排序，就可以查找得更快——好比在字典中查找单词不必一页一页翻一样。

在有序表中查找元素常常使用二分查找（Binary Search），有时也译为“折半查找”，基本思路就像是“猜数字游戏”：你在心里想一个不超过 1000 的正整数，我可以保证在 10

次之内猜到它——只要你每次告诉我猜的数比你想的大一些、小一些，或者正好猜中。

猜的方法就是“二分”。首先我猜 500，除了运气特别好正好猜中之外^①，不管你说“太大”还是“太小”，我都能把可行范围缩小一半：如果“太大”，那么答案在 1~499 之间；如果“太小”，那么答案在 501~1000 之间。只要每次选择可行区间的中点去猜，每次都可以把范围缩小一半。由于 $\log_2 1000 < 10$ ，10 次一定能猜到。

这也是二分查找的基本思路。

提示 8-9：逐步缩小范围法是一种常见的思维方法。二分查找便是基于这种思路，它遵循分治三步法，把原序列划分成元素个数尽量接近的两个子序列，然后递归查找。二分查找只适用于有序序列，时间复杂度为 $O(\log n)$ 。

尽管可以用递归实现，但一般把二分查找写成非递归的：

程序 8-5 二分查找（迭代实现）

```
int bsearch(int* A, int x, int y, int v){
    int m;
    while(x < y) {
        m = x+(y-x)/2;
        if(A[m] == v) return m;
        else if(A[m] > v) y = m;
        else x = m+1;
    }
    return -1;
}
```

上述 while 循环常常直接写在程序中。二分查找常常用在一些抽象的场合，没有数组 A，也没有要查找的 v，但是二分的思想仍然适用。

提示 8-10：二分查找一般写成非递归形式。

下面提一个有趣的问题：如果数组中有多个元素都是 v，上面的函数返回的是哪一个的下标呢？第一个？最后一个？都不是。不难看出，如果所有元素都是要找的，它返回的是中间那一个。有时，这样的结果并不是很理想，能不能求出值等于 v 的完整区间呢（由于已经排好序，相等的值会排在一起）？

下面的程序，当 v 存在时返回它出现的第一个位置。如果不存在，返回这样一个下标 i：在此处插入 v（原来的元素 A[i], A[i+1], … 全部往后移动一个位置）后序列仍然有序。

程序 8-6 二分查找求下界

```
int lower_bound(int* A, int x, int y, int v){
    int m;
    while(x < y){
```

^① 如果没有公证人，你可以不动声色地换一个数。

```

    m = x+(y-x)/2;
    if (A[m]>=v) y=m;
    else x=m+1;
}
return x;
}

```

下面来分析一下这段程序。首先，最后的返回值不仅可能是 x , $x+1$, $x+2, \dots$, $y-1$ ，还可能是 y ——如果 v 大于 $A[y-1]$ ，就只能插入这里了。这样，尽管查找区间是左闭右开区间 $[x, y)$ ，返回值的候选区间却是闭区间 $[x, y]$ 。 $A[m]$ 和 v 的各种关系所带来的影响如下。

- $A[m] = v$: 至少已经找到一个，而左边可能还有，因此区间变为 $[x, m]$ 。
- $A[m] > v$: 所求位置不可能在后面，但有可能是 m ，因此区间变为 $[x, m]$ 。
- $A[m] < v$: m 和前面都不可行，因此区间变为 $[m+1, y]$ 。

合并一下， $A[m] \geq v$ 时新区间为 $[x, m]$ ； $A[m] < v$ 时新区间为 $[m+1, y]$ 。这里有一个潜在的危险：如果 $[x, m]$ 或者 $[m+1, y]$ 和原区间 $[x, y]$ 相同，将发生死循环！幸运的是，这样的情况并不会发生，原因留给读者思考。

类似地，可以写一个 `upper_bound` 程序，当 v 存在时返回它出现的最后一个位置的后面一个位置。如果不存在，返回这样一个下标 i ：在此处插入 v （原来的元素 $A[i]$, $A[i+1]$, $\dots$ 全部往后移动一个位置）后序列仍然有序。不难得出，只需把“`if(A[m]>=v) y=m; else x=m+1;`”改成“`if(A[m]<=v) x=m+1; else y=m;`”即可。

这样，对二分查找的讨论就相对比较完整了：设 `lower_bound` 和 `upper_bound` 的返回值分别为 L 和 R ，则 v 出现的子序列为 $[L, R)$ 。这个结论当 v 不在时也成立：此时 $L=R$ ，区间为空。这里实现的 `lower_bound` 和 `upper_bound` 就是 STL 中的同名函数。

提示 8-11: 用“上下界”函数求解范围统计问题的技巧非常有用，建议读者用心体会左闭右开区间的使用方法和上下界函数的实现细节。

8.3 递归与分治

除了排序与检索外，递归还有更广泛的应用。

棋盘覆盖问题。有一个 $2^k \times 2^k$ 的方格棋盘，恰有一个方格是黑色的，其他为白色。你的任务是用包含 3 个方格的 L 型牌覆盖所有白色方格。黑色方格不能被覆盖，且任意一个白色方格不能同时被两个或更多牌覆盖。如图 8-3 所示为 L 型牌的 4 种旋转方式。

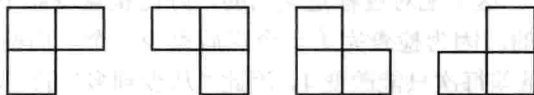


图 8-3 L 型牌

【分析】

本题的棋盘是 $2^k \times 2^k$ 的，很容易想到分治：把棋盘切为 4 块，则每一块都是 $2^{k-1} \times 2^{k-1}$ 的。有黑格的那一块可以递归解决，但其他 3 块并没有黑格子，应该怎么办呢？可以构造出一个黑格子，如图 8-4 所示。递归边界也不难得出： $k=1$ 时一块牌就够了。

循环日程表问题。 $n=2^k$ 个运动员进行网球循环赛，需要设计比赛日程表。每个选手必须与其他 $n-1$ 个选手各赛一次；每个选手一天只能赛一次；循环赛一共进行 $n-1$ 天。按此要求设计一张比赛日程表，该表有 n 行和 $n-1$ 列，第 i 行 j 列为第 i 个选手第 j 天遇到的选手。

【分析】

本题的方法有很多，递归是其中一种比较容易理解的方法。如图 8-5 所示是 $k=3$ 时的一个可行解，它是 4 块拼起来的。左上角是 $k=2$ 时的一组解，左下角是左上角每个数加 4 得到，而右上角、右下角分别由左下角、左上角复制得到。

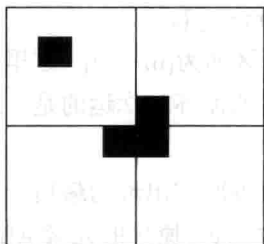


图 8-4 棋盘覆盖问题的递归解法

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

图 8-5 循环日程表问题 $k=3$ 时的解

巨人与鬼。在平面上有 n 个巨人和 n 个鬼，没有三者在同一条直线上。每个巨人需要选择一个不同的鬼，向其发送质子流消灭它。质子流由巨人发射，沿直线行进，遇到鬼后消失。由于质子流交叉是很危险的，所有质子流经过的线段不能有交点。请设计一种给巨人和鬼配对的方法。

【分析】

由于只需要一种配对方法，从直观上来说本题一定是有解的。由于每一个巨人和鬼都需要找一个目标，不妨先给“最特殊”的巨人或鬼寻找“搭档”。

考虑 y 坐标最小的点（即最低点）。如果有多个这样的点，考虑最左边的点（即其中最左边的点），则所有点的极角在范围 $[0, \pi)$ 内。不妨设它是一个巨人，然后把所有其他点按照极角从小到大的顺序排序后依次检查。

情况 1：第一个点是鬼，那么配对完成，剩下的巨人和鬼仍然是一样多，而且不会和这一条线段交叉，如图 8-6 (a) 所示。

情况 2：第一个点是巨人，那么继续检查，直到已检查的点中鬼和巨人一样多为止。找到了这个“鬼和巨人”配对区间后，只需要把此区间内的点配对，再把区域外的点配对即可，如图 8-6 (b) 所示。这个配对过程是递归的，好比棋盘覆盖中一样。会不会找不到这样的配对区间呢？不会的。因为检查完第一个点后鬼少一个，而检查完最后一个点时鬼多一个，而巨人和鬼的数量差每次只能改变 1，因此“从少到多”的过程中一定会有“一样多”的时候。

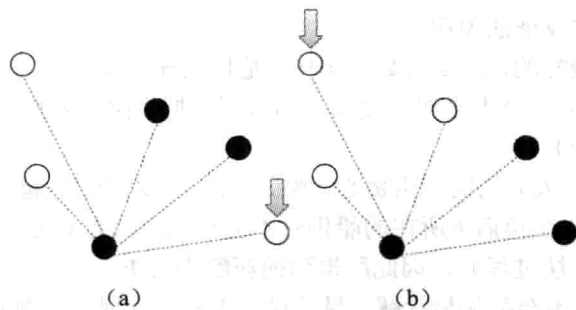


图 8-6 巨人与鬼问题

8.4 贪心法

贪心法是一种解决问题的策略。如果策略正确，那么贪心法往往是易于描述、易于实现的。本节介绍可以用贪心法解决的若干经典问题。

8.4.1 背包相关问题

最优装载问题。给出 n 个物体，第 i 个物体重量为 w_i 。选择尽量多的物体，使得总重量不超过 C 。

【分析】

由于只关心物体的数量，所以装重的没有装轻的划算。只需把所有物体按重量从小到大排序，依次选择每个物体，直到装不下为止。这是一种典型的贪心算法，它只顾眼前，但却能得到最优解。

部分背包问题。有 n 个物体，第 i 个物体的重量为 w_i ，价值为 v_i 。在总重量不超过 C 的情况下让总价值尽量高。每一个物体都可以只取走一部分，价值和重量按比例计算。

【分析】

本题在上一题的基础上增加了价值，所以不能简单地像上题那样先拿轻的（轻的可能价值也小），也不能先拿价值大的（可能它特别重），而应该综合考虑两个因素。一种直观的贪心策略是：优先拿“价值除以重量的值”最大的，直到重量和正好为 C 。

注意：由于每个物体可以只拿一部分，因此一定可以让总重量恰好为 C （或者全部拿走重量也不足 C ），而且除了最后一个以外，所有的物体要么不拿，要么拿走全部。

乘船问题。有 n 个人，第 i 个人重量为 w_i 。每艘船的最大载重量均为 C ，且最多只能乘两个人。用最少的船装载所有人。

【分析】

考虑最轻的人 i ，他应该和谁一起坐呢？如果每个人都无法和他一起坐船，则唯一的方案就是每人坐一艘船（想一想，为什么）。否则，他应该选择能和他一起坐船的人中最重的一个 j 。这样的方法是贪心的，因此它只是让“眼前”的浪费最少。幸运的是，这个贪心

策略也是对的，可以用反证法说明。

假设这样做不是最好的，那么最好方案中 i 是什么样的呢？

情况 1: i 不和任何一个人坐同一艘船，那么可以把 j 拉过来和他一起坐，总船数不会增加（而且可能会减少）。

情况 2: i 和另外一人 k 同船。由贪心策略， j 是“可以和 i 一起坐船的人”中最重的，因此 k 比 j 轻。把 j 和 k 交换后 k 所在的船仍然不会超重（因为 k 比 j 轻），而 i 和 j 所在的船也不会超重（由贪心法过程），因此所得到的新解不会更差。

由此可见，贪心法不会丢失最优解。最后说一下程序实现。在刚才的分析中，比 j 更重的人只能每人坐一艘船。这样，只需用两个下标 i 和 j 分别表示当前考虑的最轻的人和最重的人，每次先将 j 往左移动，直到 i 和 j 可以共坐一艘船，然后将 i 加 1， j 减 1，并重复上述操作。不难看出，程序的时间复杂度仅为 $O(n)$ ，是最优算法（别忘了，读入数据也需要 $O(n)$ 时间，因此无法比这个更好了）。

8.4.2 区间相关问题

选择不相交区间。数轴上有 n 个开区间 (a_i, b_i) 。选择尽量多个区间，使得这些区间两两没有公共点。

【分析】

首先明确一个问题：假设有两个区间 x, y ，区间 x 完全包含 y 。那么，选 x 是不划算的，因为 x 和 y 最多只能选一个，选 x 还不如选 y ，这样不仅区间数目不会减少，而且给其他区间留出了更多的位置。接下来，按照 b_i 从小到大的顺序给区间排序。贪心策略是：一定要选第一个区间。为什么？

现在区间已经排序成 $b_1 \leq b_2 \leq b_3 \dots$ 了，考虑 a_1 和 a_2 的大小关系。

情况 1: $a_1 > a_2$ ，如图 8-7 (a) 所示，区间 2 包含区间 1。前面已经讨论过，这种情况下一定不会选择区间 2。不仅区间 2 如此，以后所有区间中只要有一个 i 满足 $a_1 > a_i$ ， i 都不要选。在今后的讨论中，将不考虑这些区间。

情况 2: 排除了情况 1，一定有 $a_1 \leq a_2 \leq a_3 \leq \dots$ ，如图 8-7 (b) 所示。如果区间 2 和区间 1 完全不相交，那么没有影响（因此一定要选区间 1），否则区间 1 和区间 2 最多只能选一个。如果不选区间 2，黑色部分其实是没有任何影响的（它不会挡住任何一个区间），区间 1 的有效部分其实变成了灰色部分，它被区间 2 所包含！由刚才的结论，区间 2 是不能选的。依此类推，不能因为选任何区间而放弃区间 1，因此选择区间 1 是明智的。

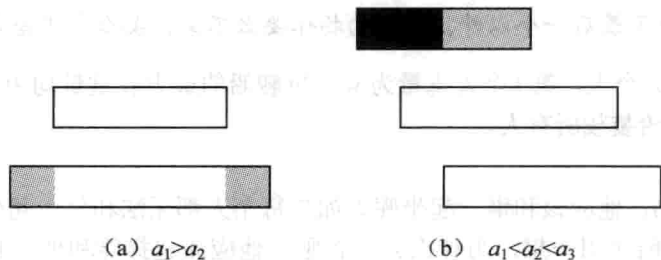


图 8-7 贪心策略图示

选择了区间 1 以后, 需要把所有和区间 1 相交的区间排除在外, 需要记录上一个被选择的区间编号。这样, 在排序后只需要扫描一次即可完成贪心过程, 得到正确结果。

区间选点问题。数轴上有 n 个闭区间 $[a_i, b_i]$ 。取尽量少的点, 使得每个区间内都至少有一个点 (不同区间内含的点可以是同一个)。

【分析】

如果区间 i 内已经有一个点被取到, 则称此区间已经被满足。受上一题的启发, 下面先讨论区间包含的情况。由于小区间被满足时大区间一定也被满足, 所以在区间包含的情况下, 大区间不需要考虑。

把所有区间按 b 从小到大排序 (b 相同时 a 从大到小排序), 则如果出现区间包含的情况, 小区间一定排在前面。第一个区间应该取哪一个点呢? 此处的贪心策略是: 取最后一个点, 如图 8-8 所示。

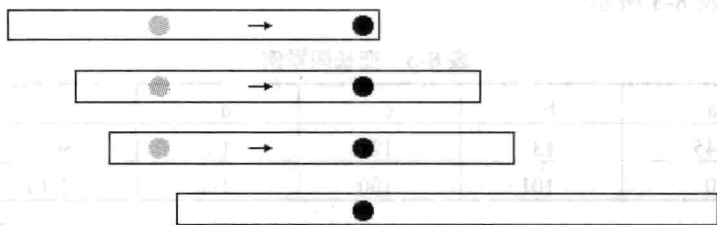


图 8-8 贪心策略

根据刚才的讨论, 所有需要考虑的区间的 a 也是递增的, 可以把它画成图 8-8 的形式。如果第一个区间不取最后一个, 而是取中间的, 如灰色点, 那么把它移动到最后一个点后, 被满足的区间增加了, 而且原先被满足的区间现在一定被满足。不难看出, 这样的贪心策略是正确的。

区间覆盖问题。数轴上有 n 个闭区间 $[a_i, b_i]$, 选择尽量少的区间覆盖一条指定线段 $[s, t]$ 。

【分析】

本题的突破口仍然是区间包含和排序扫描, 不过先要进行一次预处理。每个区间在 $[s, t]$ 外的部分都应该预先被切掉, 因为它们的存在是毫无意义的。预处理后, 在相互包含的情况下, 小区间显然不应该考虑。

把各区间按照 a 从小到大排序。如果区间 1 的起点不是 s , 无解 (因为其他区间的起点更大, 不可能覆盖到 s 点), 否则选择起点在 s 的最长区间。选择此区间 $[a_i, b_i]$ 后, 新的起点应该设置为 b_i , 并且忽略所有区间在 b_i 之前的部分, 就像预处理一样。虽然贪心策略比上题复杂, 但是仍然只需要一次扫描, 如图 8-9 所示。 s 为当前有效起点 (此前部分已被覆盖), 则应该选择区间 2。

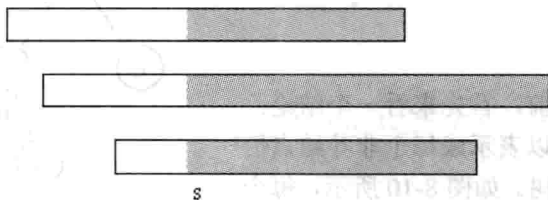


图 8-9 区间覆盖问题

8.4.3 Huffman 编码

假设某文件中只有 6 种字符: a, b, c, d, e, f, 可以用 3 个二进制位来表示, 如表 8-2 所示 (表 8-2~表 8-4 中, 频率的单位均为“千次”)。

表 8-2 各种字符的编码

字 符	a	b	c	d	e	f
频 率	45	13	12	16	9	5
编 码	000	001	010	011	100	101

这样, 一共需要 $(45+13+12+16+9+5)*3=300$ 千比特 (即二进制的位)。第二种方法是采用变长编码, 如表 8-3 所示。

表 8-3 变长码举例

字 符	a	b	c	d	e	f
频 率	45	13	12	16	9	5
编 码	0	101	100	111	1101	1100

总长度为 $1*45+3*13+3*12+3*16+4*9+4*5=224$ 千比特, 比定长码短。读者可能会说: 还可以更短, 如表 8-4 所示。

表 8-4 错误的变长码举例

字 符	a	b	c	d	e	f
频 率	45	13	12	16	9	5
编 码	0	1	00	01	10	11

总长度只有 $1*(45+13)+2*(12+16+9+5)=142$ 千比特, 不是更短吗? 可惜, 这样的编码方案是有问题的。如果收到了 001, 那么究竟是 aab、cb, 还是 ad? 换句话说, 这样的编码有歧义, 因为其中一个字符的编码是另一个码的前缀 (prefix)。表 8-3 所示的码没有这样的情况, 任何一个编码都不是另一个的前缀。这里把满足这样性质的编码称为前缀码 (Prefix Code)。下面正式叙述编码问题。

最优编码问题。给出 n 个字符的频率 c_i , 给每个字符赋予一个 01 编码串, 使得任意一个字符的编码不是另一个字符编码的前缀, 而且编码后总长度 (每个字符的频率与编码长度乘积的总和) 尽量小。

【分析】

在解决这个问题之前, 首先来看一个结论: 任何一个前缀编码都可以表示成每个非叶结点恰好有两个子结点的二叉树。如图 8-10 所示, 每个非叶结点与左子结点的边上写 1, 与右子结点的

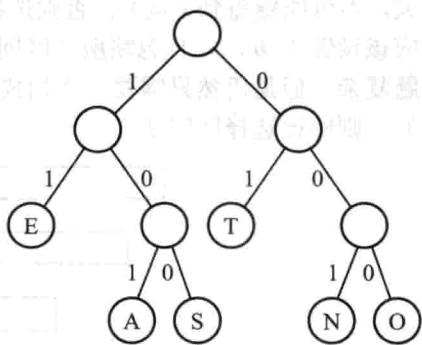


图 8-10 前缀码的二叉树表示

边上写0。

每个叶子对应一个字符，编码为从根到该叶子的路径上的01序列。在图8-10中，N的编码为001，而E的编码为11。为了证明在一般情况下，都可以用这样的二叉树来表示最优前缀码，需要证明两个结论。

结论1： n 个叶子的二叉树一定对应一个前缀码。如果编码 a 为编码 b 的前缀，则 a 所对应的结点一定为 b 所对应结点的祖先。而两个叶子不会有祖先后代的关系。

结论2：最优前缀码一定可以写成二叉树。逐个字符构造即可。每拿到一个编码，都可以构造出从根到叶子的一条路径，沿着已有结点走，创建不存在的结点。这样得到的二叉树不可能有单子结点，因为如果存在，只要用这个子结点代替父结点，得到的仍然是前缀码，且总长度更短。

接下来的问题就变为：如何构造一棵最优的编码树。

Huffman 算法：把每个字符看作一个单结点子树放在一个树集合中，每棵子树的权值等于相应字符的频率。每次取权值最小的两棵子树合并成一棵新树，并重新放到集合中。新树的权值等于两棵子树权值之和。

下面分两步证明算法的正确性。

结论1：设 x 和 y 是频率最小的两个字符，则存在前缀码使得 x 和 y 具有相同码长，且仅有最后一位编码不同。换句话说，**第一步贪心法选择保留最优解。**

证明：假设深度最大的结点为 a ，则 a 一定有一个兄弟 b 。不妨设 $f(x) \leq f(y)$ ， $f(a) \leq f(b)$ ，则 $f(x) \leq f(a)$ ， $f(y) \leq f(b)$ 。如果 x 不是 a ，则交换 x 和 a ；如果 y 不是 b ，则交换 y 和 b 。这样得到的新编码树不会比原来的差。

结论2：设 T 是加权字符集 C 的最优编码树， x 和 y 是树 T 中两个叶子，且互为兄弟结点， z 是它们的父结点。若把 z 看成具有频率 $f(z)=f(x)+f(y)$ 的字符，则树 $T' = T - \{x, y\}$ 是字符集 $C' = C - \{x, y\} \cup \{z\}$ 的一棵最优编码树。换句话说，原问题的最优解包含子问题的最优解。

证明：设 T' 的编码长度为 L ，其中字符 $\{x, y\}$ 的深度为 h ，则把字符 $\{x, y\}$ 拆成两个后，长度变为 $L - (f(x) + f(y)) \cdot h + (f(x) + f(y)) \cdot (h + 1) = L + f(x) + f(y)$ 。因此 T' 必须是 C' 的最优编码树， T 才是 C 的最优编码树。

结论1通常称为贪心选择性质，结论2通常称为最优子结构性质。根据这两个结论，Huffman算法正确。在程序实现上，可以先按照频率把所有字符排序成表 P ，然后创建一个新结点队列 Q ，在每次合并两个结点后把新结点放到队列 Q 中。由于后合并的频率和一定比先合并的频率和大，因此 Q 内的元素是有序的。类似有序表的合并过程，每次只需要检查 P 和 Q 的首元素即可找到频率最小的元素，时间复杂度为 $O(n)$ 。算上排序，总时间复杂度为 $O(n \log n)$ 。

8.5 算法设计与优化策略

本节是本章的重点，也是“基础篇”中第一个贴近竞赛的小节。竞赛中常用的算法设计方法有很多，本节列举一些较为经典的专题，以供读者学习。

构造法。很多时候可以通过“直接构造解”的方法来解决问题。这是最没有规律可循的一种方法，也是最考验“真功夫”的一种方法。

例题 8-1 煎饼 (Stacks of Flapjacks, UVa120)

有一叠煎饼正在锅里。煎饼共有 n ($n \leq 30$) 张，每张都有一个数字，代表它的大小，如图 8-11 所示。厨师每次可以选择一个数 k ，把从锅底开始数第 k 张上面的煎饼全部翻过来，即原来在上面的煎饼现在到了下面。例如，图 8-11 (a)，依次执行操作 3 次后得到图 8-11 (c) 的情况。

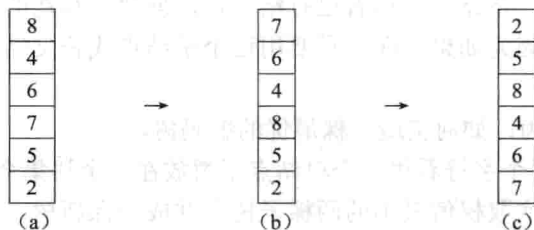


图 8-11 煎饼问题示意图

设计一种方法使得所有煎饼按照从小到大排序（最上面的煎饼最小）。输入时，各个煎饼按照从上到下的顺序给出。例如，上面的例子输入为 8, 4, 6, 7, 5, 2。

【分析】

这道题目要求排序，但是基本操作却是“颠倒一个连续子序列”。不过没有关系，我们还是可以按照选择排序的思想，以从大到小的顺序依次把每个数排到正确的位置。方法是先翻到最上面，然后翻到正确的位置。由于是按照从大到小的顺序处理，当处理第 i 大的煎饼时，是不会影响到第 1, 2, 3, ..., $i-1$ 大的煎饼的（它们已经正确地翻到了煎饼堆底部的 $i-1$ 个位置上）。

例题 8-2 联合国大楼 (Building for UN, ACM/ICPC NEERC 2007, UVa1605)

你的任务是设计一个包含若干层的联合国大楼，其中每层都是一个等大的网格。有若干国家需要在联合国大楼里办公，你需要把每个格子分配给一个国家，使得任意两个不同的国家都有一对相邻的格子（要么是同层中有公共边的格子，要么是相邻层的同一个格子）。你设计的大厦最多不能超过 1000000 个格子。

输入国家的个数 n ($n \leq 50$)，输出大楼的层数 H 、每层楼的行数 W 和列数 L ，然后是每层楼的平面图。不同国家用不同的大小写字母表示。例如， $n=4$ 的一组解是 $H=W=L=2$ ，

第一层是 $\begin{smallmatrix} AB \\ CC \end{smallmatrix}$ ，第二层是 $\begin{smallmatrix} ZZ \\ ZZ \end{smallmatrix}$ 。

【分析】

本题的限制非常少，层数、行数和列数都可以任选。正因为如此，本题的解法非常多。其中有一种方法比较值得探讨：一共只有两层，每层都是 $n \times n$ 的，第一层第 i 行全是国家 i ，第二层第 j 列全是国家 j 。请读者自己验证它是如何满足题目要求的。

中途相遇法。这是一种特殊的算法，大体思路是从两个不同的方向来解决问题，最终“汇集”到一起。第 7 章中提到的“双向广度优先搜索”方法就有一点中途相遇法的味道。

下面再举一个更为直接的例子。

例题 8-3 和为 0 的 4 个值 (4 Values Whose Sum is Zero, ACM/ICPC SWERC 2005, UVa 1152)

给定 4 个 n ($1 \leq n \leq 4000$) 元素集合 A, B, C, D , 要求分别从中选取一个元素 a, b, c, d , 使得 $a+b+c+d=0$ 。问: 有多少种选法?

例如, $A=\{-45, -41, -36, 26, -32\}$, $B=\{22, -27, 53, 30, -38, -54\}$, $C=\{42, 56, -37, -75, -10, -6\}$, $D=\{-16, 30, 77, -46, 62, 45\}$, 则有 5 种选法: $(-45, -27, 42, 30)$, $(26, 30, -10, -46)$, $(-32, 22, 56, -46)$, $(-32, 30, -75, 77)$, $(-32, -54, 56, 30)$ 。

【分析】

最容易想到的算法就是写一个四重循环枚举 a, b, c, d , 看看加起来是否等于 0, 时间复杂度为 $O(n^4)$, 超时。一个稍好的方法是枚举 a, b, c , 则只需要在集合 D 里找找是否有元素 $-a-b-c$, 如果存在, 则方案加 1。如果排序后使用二分查找, 时间复杂度为 $O(n^3 \log n)$ 。

把刚才的方法加以推广, 就可以得到一个更快的算法: 首先枚举 a 和 b , 把所有 $a+b$ 记录下来放在一个有序数组或者 STL 的 map 里, 然后枚举 c 和 d , 查一查 $-c-d$ 有多少种方法写成 $a+b$ 的形式。两个步骤都是 $O(n^2 \log n)$, 总时间复杂度也是 $O(n^2 \log n)$ 。

需要注意的是: 由于本题数据规模较大, 有些时间复杂度为 $O(n^2 \log n)$ 但常数较大的算法在 UVa 上会超时 (例如使用 STL 中的 map 就很容易超时)。笔者推荐的高效实现方法是把所有 $a+b$ 放到一个自己实现的哈希表中, 但建议读者自行尝试不同算法以及实现方法, 这样可以对它们的实际运行效率有一个更直观的认识。

问题分解。有时候可以把一个复杂的问题分解成若干个独立的简单问题, 并加以求解。下面就是一个很好的例子。

例题 8-4 传说中的车 (Fabled Rooks, UVa 11134)

你的任务是在 $n \times n$ 的棋盘上放 n ($n \leq 5000$) 个车, 使得任意两个车不相互攻击, 且第 i 个车在一个给定的矩形 R_i 之内。用 4 个整数 xl_i, yl_i, xr_i, yr_i ($1 \leq xl_i \leq xr_i \leq n, 1 \leq yl_i \leq yr_i \leq n$) 描述第 i 个矩形, 其中 (xl_i, yl_i) 是左上角坐标, (xr_i, yr_i) 是右下角坐标, 则第 i 个车的位置 (x, y) 必须满足 $xl_i \leq x \leq xr_i, yl_i \leq y \leq yr_i$ 。如果无解, 输出 IMPOSSIBLE; 否则输出 n 行, 依次为第 $1, 2, \dots, n$ 个车的坐标。

【分析】

两个车相互攻击的条件是处于同一行或者同一列, 因此不相互攻击的条件就是不在同一行, 也不在同一列。可以看出: 行和列是无关的, 因此可以把原题分解成两个一维问题。在区间 $[1 \sim n]$ 内选择 n 个不同的整数, 使得第 i 个整数在闭区间 $[n1_i, n2_i]$ 内。是不是很像前面讲过的贪心法题目? 这也是一个不错的练习, 具体解法留给读者思考。

等价转换。与其说这是一种算法设计方法, 还不如说是一种思维方式, 可以帮助选手理清思路, 甚至直接得到问题的解决方案。

例题 8-5 Gergovia 的酒交易 (Wine trading in Gergovia, UVa 11054)

直线上有 n ($2 \leq n \leq 100000$) 个等距的村庄, 每个村庄要么买酒, 要么卖酒。设第 i 个村庄对酒的需求为 a_i ($-1000 \leq a_i \leq 1000$), 其中 $a_i > 0$ 表示买酒, $a_i < 0$ 表示卖酒。所有村庄供需平衡, 即所有 a_i 之和等于 0。

把 k 个单位的酒从一个村庄运到相邻村庄需要 k 个单位的劳动力。计算最少需要多少劳动力可以满足所有村庄的需求。输出保证在 64 位带符号整数的范围内。

【分析】

考虑最左边的村庄。如果需要买酒，即 $a_1 > 0$ ，则一定有劳动力从村庄 2 往左运给村庄 1，而不管这些酒是从哪里来的（可能就是村庄 2 产的，也可能是更右边的村庄运到村庄 2 的）。这样，问题就等价于只有村庄 2~ n ，且第 2 个村庄的需求为 $a_1 + a_2$ 的情形。不难发现， $a_i < 0$ 时这个推理也成立（劳动力同样需要 $|a_i|$ 个单位）。代码如下：

```
int main() {
    int n;
    while(cin >> n && n) {
        long long ans = 0, a, last = 0;
        for(int i = 0; i < n; i++) {
            cin >> a;
            ans += abs(last);
            last += a;
        }
        cout << ans << "\n";
    }
    return 0;
}
```

扫描法。扫描法类似于一种带有顺序的枚举法。例如，从左到右考虑数组的各个元素，也可以说从左到右“扫描”。它和普通枚举法的重要区别是：扫描法往往在枚举时维护一些重要的量，从而简化计算。

例题 8-6 两亲性分子 (Amphiphilic Carbon Molecules, ACM/ICPC Shanghai 2004, UVa1606)

平面上有 n ($n \leq 1000$) 个点，每个点为白点或者黑点。现在需放置一条隔板，使得隔板一侧的白点数加上另一侧的黑点数总数最大。隔板上的点可以看作是在任意一侧。

【分析】

不妨假设隔板一定经过至少两个点（否则可以移动隔板使其经过两个点，并且总数不会变小），则最简单的想法是：枚举两个点，然后输出两侧黑白点的个数。枚举量是 $O(n^2)$ ，再加上统计的 $O(n)$ ，总时间复杂度为 $O(n^3)$ 。

可以先枚举一个基准点，然后将一条直线绕这个点旋转。每当直线扫过一个点，就可以动态修改（这就是“维护”）两侧的点数。在直线旋转“一圈”的过程中，每个点至多被扫描到两次，如图 8-12 所示。因此这个过程的复杂度为 $O(n)$ 。由于扫描之前要将所有点按照相对基准点的极角排序，再加上基准点的 n 种取法，算法的总时间复杂度为 $O(n^2 \log n)$ 。

需要注意的是，本题存在多点共线的情况，如果用反三角函数计算极角，然后判断极角是否相同的话，很容易产生精度误差。

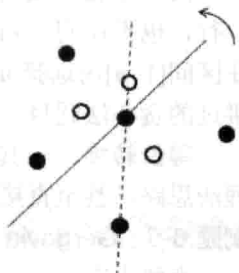


图 8-12 枚举基准点

应该把极角相等的条件进行化简（或者直接使用叉积），只使用整数运算进行判断。

滑动窗口。滑动窗口非常有特色，下面的例子很好地说明了这一点。

例题 8-7 唯一的雪花 (Unique snowflakes, UVa 11572)

输入一个长度为 n ($n \leq 10^6$) 的序列 A ，找到一个尽量长的连续子序列 $A_L \sim A_R$ ，使得该序列中没有相同的元素。

【分析】

假设序列元素从 0 开始编号，所求连续子序列的左端点为 L ，右端点为 R 。首先考虑起点 $L=0$ 的情况。可以从 $R=0$ 开始不断增加 R ，相当于把所求序列的右端点往右延伸。当无法延伸（即 $A[R+1]$ 在子序列 $A[L \sim R]$ 中出现过）时，只需增大 L ，并且继续延伸 R 。既然当前的 $A[L \sim R]$ 是可行解， L 增大之后必然还是可行解，所以不必减少 R ，继续增大即可。

不难发现这个算法是正确的，不过真正有意思的是算法的时间复杂度。暂时先不考虑“判断是否可以延伸”这个部分，每次要么把 R 加 1，要么把 L 加 1，而 L 和 R 最多从 0 增加到 $n-1$ ，所以指针增加的次数是 $O(n)$ 的。

最后考虑“判断是否可以延伸”这个部分。比较容易想到的方法是用一个 STL 的 `set`，保存 $A[L \sim R]$ 中元素的集合，当 R 增大时判断 $A[R+1]$ 是否在 `set` 中出现，而 R 加 1 时把 $A[R+1]$ 插入到 `set` 中， $L+1$ 时把 $A[L]$ 从 `set` 中删除。因为 `set` 的插入删除和查找都是 $O(\log n)$ 的，所以这个算法的时间复杂度为 $O(n \log n)$ 。代码如下：

```
#include<cstdio>
#include<set>
#include<algorithm>
using namespace std;

const int maxn = 1000000 + 5;
int A[maxn];

int main() {
    int T, n;
    scanf("%d", &T);
    while(T--) {
        scanf("%d", &n);
        for(int i = 0; i < n; i++) scanf("%d", &A[i]);

        set<int> s;
        int L = 0, R = 0, ans = 0;
        while(R < n) {
            while(R < n && !s.count(A[R])) s.insert(A[R++]);
            ans = max(ans, R - L);
            s.erase(A[L++]);
        }
    }
}
```

```

    printf("%d\n", ans);
}
return 0;
}

```

另一个方法是用一个 `map` 求出 `last[i]`，即下标 i 的“上一个相同元素的下标”。例如，输入序列为 3 2 4 1 3 2 3，当前区间是 $[1, 3]$ （即元素 2, 4, 1），是否可以延伸呢？下一个数是 $A[5]=3$ ，它的“上一个相同位置”是下标 0 ($A[0]=3$)，不在区间中，因此可以延伸。`map` 的所有操作都是 $O(\log n)$ 的，但后面所有操作的时间复杂度均为 $O(1)$ ，总时间复杂度也是 $O(n \log n)$ 。代码如下：

```

#include<cstdio>
#include<map>
using namespace std;

const int maxn = 1000000 + 5;
int A[maxn], last[maxn];
map<int, int> cur;

int main() {
    int T, n;
    scanf("%d", &T);
    while(T--) {
        scanf("%d", &n);
        cur.clear();
        for(int i = 0; i < n; i++) {
            scanf("%d", &A[i]);
            if(!cur.count(A[i])) last[i] = -1;
            else last[i] = cur[A[i]];
            cur[A[i]] = i;
        }

        int L = 0, R = 0, ans = 0;
        while(R < n) {
            while(R < n && last[R] < L) R++;
            ans = max(ans, R - L);
            L++;
        }
        printf("%d\n", ans);
    }
    return 0;
}

```

本题非常经典，请读者仔细品味。

使用数据结构。数据结构往往可以在不改变主算法的前提下提高运行效率，具体做法可能千差万别，但思路却是有规律可循的。下面先介绍一个经典问题。

输入正整数 k 和一个长度为 n 的整数序列 $A_1, A_2, A_3, \dots, A_n$ 。定义 $f(i)$ 表示从元素 i 开始的连续 k 个元素的最小值，即 $f(i) = \min\{A_i, A_{i+1}, \dots, A_{i+k-1}\}$ 。要求计算 $f(1), f(2), f(3), \dots, f(n-k+1)$ 。例如，对于序列 5, 2, 6, 8, 10, 7, 4, $k=4$ ，则 $f(1)=2, f(2)=2, f(3)=6, f(4)=4$ 。

【分析】

如果使用定义，每个 $f(i)$ 都需要 $O(k)$ 时间计算，总时间复杂度为 $O((n-k)k)$ ，太大了。那么换一个思路：计算 $f(1)$ 时，需要求 k 个元素的最小值——这是一个“窗口”。计算 $f(2)$ 时，这个窗口向右滑动了一个位置，计算 $f(3)$ 和 $f(4)$ 时，窗口各滑动了一个位置，如图 8-13 所示。



图 8-13 窗口滑动

因此，这个问题称为**滑动窗口的最小值问题**。窗口在滑动的过程中，窗口中的元素“出去”了一个，又“进来”了一个。借用数据结构中的术语，窗口往右滑动时需要删除一个元素，然后插入一个元素，还需要取最小值。这不就是优先队列吗？第 5 章中曾经介绍过用 STL 集合实现一个支持删除任意元素的优先队列。因为窗口中总是有 k 个元素，插入、删除、取最小值的时间复杂度均为 $O(\log k)$ 。这样，每次把窗口滑动时都需要 $O(\log k)$ 的时间，一共滑动 $n-k$ 次，因此总时间复杂度为 $O((n-k)\log k)$ 。

其实还可以做得更好。假设窗口中有两个元素 1 和 2，且 1 在 2 的右边，会怎样？这意味着 2 在离开窗口之前永远不可能成为最小值。换句话说，这个 2 是无用的，应当及时删除。当删除无用元素之后，滑动窗口中的有用元素从左到右是递增的。为了叙述方便，习惯上称其为**单调队列**。在单调队列中求最小值很容易：队首元素就是最小值。

当窗口滑动时，首先要删除滑动前窗口的最左边元素（如果是有用元素），然后把新元素加入单调队列。注意，比新元素大的元素都变得无用了，应当从右往左删除。如图 8-14 所示是滑动窗口的 4 个位置所对应的单调队列。

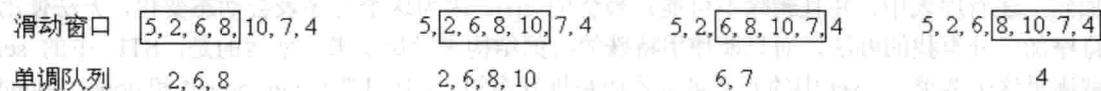


图 8-14 滑动窗口对应的单调队列

单调队列和普通队列有些不同，因为右端既可以插入又可以删除，因此在代码中通常用一个数组和 `front`、`rear` 两个指针来实现，而不是用 STL 中的 `queue`。如果一定要用 STL，则需要用双端队列（即两端都可以插入和删除），即 `deque`。

尽管插入元素时可能会删除多个元素，但因为每个元素最多被删除一次，所以总的时间复杂度仍为 $O(n)$ ，达到了理论下界（因为至少需要 $O(n)$ 的时间来检查每个元素）。

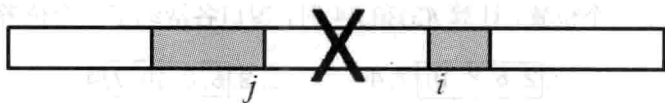
下面这道例题更加复杂，但思路是一样的：先排除一些干扰元素（无用元素），然后把有用的元素组织成易于操作的数据结构。

例题 8-8 防线 (Defense Lines, ACM/ICPC CERC 2010, UVa1471)

给一个长度为 n ($n \leq 200000$) 的序列, 你的任务是删除一个连续子序列, 使得剩下的序列中有一个长度最大的连续递增子序列。例如, 将序列 $\{5, 3, 4, 9, 2, 8, 6, 7, 1\}$ 中的 $\{9, 2, 8\}$ 删除, 得到的序列 $\{5, 3, 4, 6, 7, 1\}$ 中包含一个长度为 4 的连续递增子序列 $\{3, 4, 6, 7\}$ 。序列中每个数均为不超过 10^9 的正整数。

【分析】

为了方便叙述, 下面用 L 序列表示“连续递增子序列”。删除一个子序列之后, 得到的最长 L 序列应该是由两个序列拼起来的, 如图 8-15 所示。

图 8-15 最长序列 L

最容易想到的算法是枚举 j 和 i (前提是 $A[j] < A[i]$, 否则拼不起来), 然后分别往左和往右数一数最远能延伸到哪里。枚举量为 $O(n^2)$, 而“数一数”的时间复杂度为 $O(n)$, 因此总时间复杂度为 $O(n^3)$ 。

加上一个预处理, 就能避免“数一数”这个过程, 从而把时间复杂度降为 $O(n^2)$ 。设 $f(i)$ 为以第 i 个元素开头的最长 L 序列长度, $g(i)$ 为以第 i 个元素结尾的最长 L 序列长度, 则不难在 $O(n)$ 时间内求出 $f(i)$ 和 $g(i)$, 然后枚举完 j 和 i 之后, 最长 L 序列的长度就是 $g(j) + f(i)$ 。

还可以做得更好: 只枚举 i , 不枚举 j , 而是用其他方法快速找一个 $j < i$, 使得 $A[j] < A[i]$, 且 $g(j)$ 尽量大。如何快速找到呢? 首先要排除一些肯定不是最优值的 j 。例如, 若有 j' 满足 $A[j'] \leq A[j]$ 且 $g(j') > g(j)$, 则 j 肯定不满足条件, 因为 j' 不仅是一个更长的 L 序列的末尾, 而且它更容易拼成。

这样, 把所有“有保留价值”的 j 按照 $A[j]$ 从小到大排成一个有序表 (根据刚才的结论, $A[j]$ 相同的 j 只保留一个), 则 g 也会是从小到大排列。那么用二分查找找到满足 $A[j] < A[i]$ 的最大的 $A[j]$, 则它对应的 $g(j)$ 也是最大的。

不过这个方法只有当 i 固定时才有效。实际上每次计算完一个 $g(i)$ 之后, 还要把这个 $A[i]$ 加到上述有序表中, 并且删除不可能是最优的 $A[j]$ 。因为这个有序表会动态变化, 无法使用排序加二分查找的办法, 而只能使用特殊的数据结构来满足要求。幸运的是, STL 中的 `set` 就满足这个要求——`set` 中的元素可以看成是排好序的, 而且自带 `lower_bound` 和 `upper_bound` 函数, 作用和之前讨论过的一样。

为了方便起见, 此处用二元组 $(A[j], g(j))$ 表示这些“有保留价值”的东西, 如 $(10, 4)$, $(20, 8)$, $(30, 15)$, $(40, 18)$, $(50, 30)$, 并且以 $A[j]$ 为关键字放在一个 STL 集合中。对于固定的 i , 不难用 `lower_bound` 满足 $A[j] < A[i]$ 的最大 $A[j]$, 以及对应的 $g(j)$, 真正复杂的是这个集合本身的更新, 即前面提到的“每次计算完一个 $g(i)$ 之后”需要做的事情。

假设已经计算出一个 $g(i) = 6$, 且 $A[i] = 25$, 接下来会发生什么事情? 首先把 $(25, 6)$ 插入集合中, 然后检查它的前一个元素 $(20, 8)$ 。由于 $20 < 25$, $8 > 6$, $(25, 6)$ 是不应该保留的。但如果插入的是 $(25, 20)$, 情况就完全不同了: 不仅 $(25, 20)$ 需要保留, 而且还要删除 $(30, 15)$ 和 $(40, 18)$ 。一般地, 插入任何一个二元组时首先应找到其插入位置, 根据它前一个元素判断是否需要

保留。如果需要保留,再往后遍历,删除所有不再需要保留的元素。因为所有元素至多被删除一次,而查找、插入和删除的时间复杂度均为 $O(\log n)$,所以消耗在 STL 集合上的总时间复杂度为 $O(n \log n)$ ^①。

数形结合。数形结合是一种相对高级的算法设计策略,虽有一定规律可循,但仍然灵活多变。通过下面的例题,读者可对其中的奥妙了解一二。

例题 8-9 平均值 (Average, Seoul 2009, UVa1451)

给定一个长度为 n 的 01 串,选一个长度至少为 L 的连续子串,使得子串中数字的平均值最大。如果有多解,子串长度应尽量小;如果仍有多解,起点编号尽量小。序列中的字符编号为 $1 \sim n$,因此 $[1, n]$ 就是完整的字符串。 $1 \leq n \leq 100000$, $1 \leq L \leq 1000$ 。

例如,对于如下长度为 17 的序列 0010101101101010,如果 $L=7$,最大平均值为 $6/8$ (子序列为 $[7, 14]$,其长度为 8);如果 $L=5$,子序列 $[7, 11]$ 的平均值最大,为 $4/5$ 。

【分析】

先求前缀和 $S_i = A_1 + A_2 + \dots + A_i$ (规定 $S_0 = 0$),然后令点 $P_i = (i, S_i)$,则子序列 $i \sim j$ 的平均值为 $(S_j - S_i) / (j - i + 1)$,也就是直线 $P_i P_j$ 的斜率。这样可得到主算法:从小到大枚举 t ,快速找到 $t' \leq t - L$,使得 $P_{t'} P_t$ 斜率最大。注意题目中的 A_i 都是 0 或 1,因此每个 P_i 和上一个 P_{i-1} 相比,都是 x 加 1, y 不变或者加 1。

对于给定的 t ,要找的点 $P_{t'}$ 在 P_t 的左边。假设有 3 个候选点 P_i, P_j, P_k ,下标满足 $i < j < k < t$,并且 3 个点成上凸形状(P_j 为上凸点)。假设 P_t 的 x 坐标为 x_0 ,根据定义, P_t 的 y 坐标一定不小于 P_k 的 y 坐标,因此 P_t 一定位于 A, B, C 3 条线段/射线之一,如图 8-16 所示。

□ 当 P_t 在射线 A 上时, P_k 比 P_j 好(即 $P_k P_t$ 的斜率比 $P_j P_t$ 的斜率大,后同)。

□ 当 P_t 在线段 B 上时, P_i 比 P_j 好。

□ 当 P_t 在线段 C 上时, P_i 和 P_k 都比 P_j 好。

换句话说,只要出现上凸的情况,上凸点一定可以忽略。

假设已经有了一些下凸点,现在又加入了一个点,可能会使一些已有的点变为上凸点,这时就应当将这些上凸点删除。由于被删除的点总是原来的下凸点中最右边的若干个连续点,所以可以用栈来实现,如图 8-17 所示。

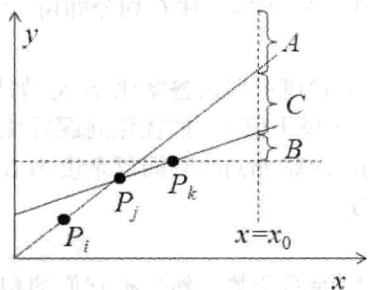


图 8-16 平均值问题示意图

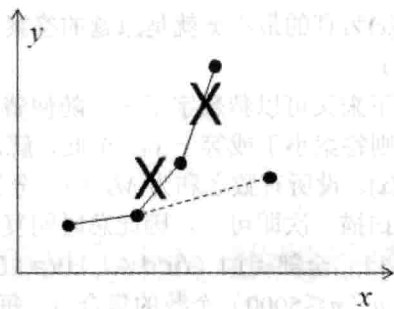


图 8-17 下凸点

^① 有一个办法可以避开 STL 集合:用一个数组保存每个 $g(j)$ 对应的最小的 $A[j]$ 。细节请参考《算法竞赛入门经典——训练指南》第 1 章中介绍的 LIS 的 $O(n \log n)$ 算法。

得到下凸线之后, 对于任何一个点 P_t 来说, 最优点 P_t 都在切点, 如图 8-18 所示。

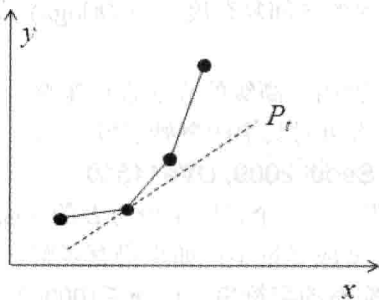


图 8-18 最优点 P_t

如何求切点呢? 随着 t 的增大, 斜率也是越来越大, 所以每次求出的 t' 只会增大, 不会减小。因此每次增加到斜率变小时停下来即可。时间复杂度为 $O(n)$ 。

8.6 竞赛题目选讲

例题 8-10 抄书 (Copying Books, UVa 714)

把一个包含 m 个正整数的序列划分成 k 个 ($1 \leq k \leq m \leq 500$) 非空的连续子序列, 使得每个正整数恰好属于一个序列。设第 i 个序列的各数之和为 $S(i)$, 你的任务是让所有 $S(i)$ 的最大值尽量小。例如, 序列 1 2 3 2 5 4 划分成 3 个序列的最优方案为 1 2 3 | 2 5 | 4, 其中 $S(1)$ 、 $S(2)$ 、 $S(3)$ 分别为 6、7、4, 最大值为 7; 如果划分成 1 2 | 3 2 | 5 4, 则最大值为 9, 不如刚才的好。每个整数不超过 10^7 。如果有多解, $S(1)$ 应尽量小。如果仍然有多解, $S(2)$ 应尽量小, 依此类推。

【分析】

“最大值尽量小”是一种很常见的优化目标。下面考虑一个新的问题: 能否把输入序列划分成 m 个连续的子序列, 使得所有 $S(i)$ 均不超过 x ? 将这个问题的答案用谓词 $P(x)$ 表示, 则让 $P(x)$ 为真的最小 x 就是原题的答案。 $P(x)$ 并不难计算, 每次尽量往右划分即可 (想一想, 为什么)。

接下来又可以猜数字了——随便猜一个 x_0 , 如果 $P(x_0)$ 为假, 那么答案比 x_0 大; 如果 $P(x_0)$ 为真, 则答案小于或等于 x_0 。至此, 解法已经得出: 二分最小值 x , 把优化问题转化为判定问题 $P(x)$ 。设所有数之和为 M , 则二分次数为 $O(\log M)$, 计算 $P(x)$ 的时间复杂度为 $O(n)$ (从左到右扫描一次即可), 因此总时间复杂度为 $O(n \log M)$ 。

例题 8-11 全部相加 (Add All, UVa 10954)

有 n ($n \leq 5000$) 个数的集合 S , 每次可以从 S 中删除两个数, 然后把它们的和放回集合, 直到剩下一个数。每次操作的开销等于删除的两个数之和, 求最小总开销。所有数均小于 10^5 。

【分析】

这不就是 Huffman 编码的建立过程吗? 因为 n 比较小, 还可以采用一种更容易写的方

法——使用一个优先队列。

```
#include<cstdio>
#include<queue>
using namespace std;

int main() {
    int n, x;
    while(scanf("%d", &n) == 1 && n) {
        priority_queue<int, vector<int>, greater<int> > q;
        for(int i = 0; i < n; i++) { scanf("%d", &x); q.push(x); }
        int ans = 0;
        for(int i = 0; i < n-1; i++) {
            int a = q.top(); q.pop();
            int b = q.top(); q.pop();
            ans += a+b;
            q.push(a+b);
        }
        printf("%d\n", ans);
    }
    return 0;
}
```

例题 8-12 奇怪的气球膨胀 (Erratic Expansion, UVa12627)

一开始有一个红气球。每小时后，一个红气球会变成 3 个红气球和一个蓝气球，而一个蓝气球会变成 4 个蓝气球，如图 8-19 所示分别是经过 0, 1, 2, 3 小时后的情况。经过 k 小时后，第 $A \sim B$ 行一共有多少个红气球？例如， $k=3$ ， $A=3$ ， $B=7$ ，答案为 14。

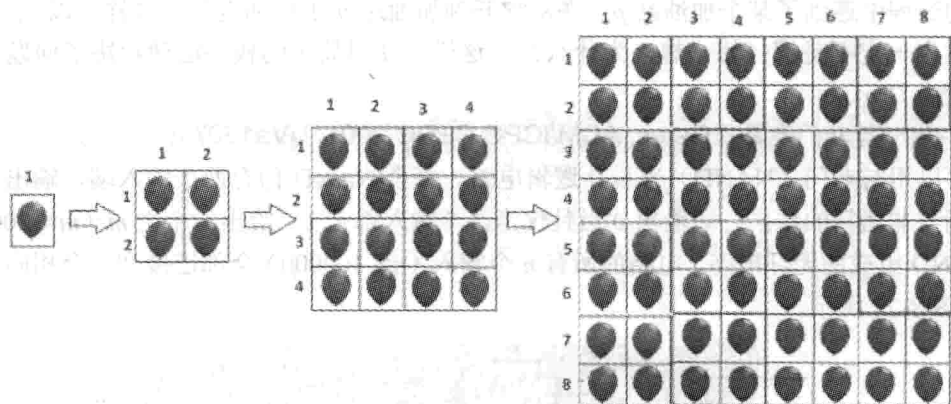


图 8-19 奇怪的气球膨胀示意图

【分析】

如图 8-20 所示， k 小时的情况由 4 个 $k-1$ 小时的情况拼成，其中右下角全是蓝气球，不用考虑。剩下的 3 个部分有一个共同点：都是前 $k-1$ 小时后“最下面若干行”或者“最

上面若干行”的红气球总数。

具体来说，设 $f(k, i)$ 表示 k 小时之后最上面 i 行的红气球总数， $g(k, i)$ 表示 k 小时之后最下面 i 行的红气球总数（规定 $i \leq 0$ 时 $f(k, i) = g(k, i) = 0$ ），则所求答案为 $f(k-1, B-2^{k-1}) + 2 * g(k-1, 2^{k-1}-A+1)$ 。

如何计算 $f(k, i)$ 和 $g(k, i)$ 呢？以 $g(k, i)$ 为例，下面分两种情况进行讨论，如图 8-21 所示。

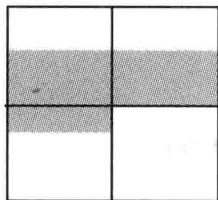


图 8-20 k 小时的情况

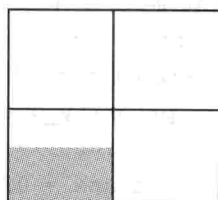
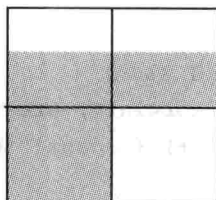


图 8-21 计算 $g(k, i)$

如果 $i \geq 2^{k-1}$ ，则 $g(k, i) = 2g(k-1, i-2^{k-1}) + c(k)$ ，否则 $g(k, i) = g(k-1, i)$ 。其中， $c(k)$ 表示 k 小时后红气球的总数，满足递推式 $c(k) = 3c(k-1)$ ，而 $c(0) = 1$ ，因此 $c(k) = 3^k$ 。

不管是哪种情况， $g(k, i)$ 都可以直接转化为 $k-1$ 的情况，因此 $g(k, i)$ 的计算时间为 $O(k)$ 。类似地， $f(k, i)$ 的计算时间也是 $O(k)$ ，因此本题的总时间复杂度为 $O(k)$ 。

例题 8-13 环形跑道（Just Finish it up, UVa 11093）

环形跑道上有 n ($n \leq 100000$) 个加油站，编号为 $1 \sim n$ 。第 i 个加油站可以加油 p_i 加仑。从加油站 i 开到下一站需要 q_i 加仑汽油。你可以选择一个加油站作为起点，初始油箱为空（但可以立即加油）。你的任务是选择一个起点，使得可以走完一圈后回到起点。假定油箱中的油量没有上限。如果无解，输出 Not possible，否则输出可以作为起点的最小加油站编号。

【分析】

考虑 1 号加油站，直接模拟判断它是否为解。如果是，直接输出；如果不是，说明在模拟的过程中遇到了某个加油站 p ，在从它开到加油站 $p+1$ 时油没了。这样，以 2, 3, ..., p 为起点也一定不是解（想一想，为什么）。这样，使用简单的枚举法便解决了问题，时间复杂度为 $O(n)$ 。

例题 8-14 与非门电路（Gates, ACM/ICPC CERC 2001, UVa1607）

可以用与非门（NAND）来设计逻辑电路。每个 NAND 门有两个输入端，输出为两个输入端与非运算的结果。即输出 0 当且仅当两个输入都是 1。给出一个由 m ($m \leq 200000$) 个 NAND 组成的无环电路，电路的所有 n 个输入 ($n \leq 100000$) 全部连接到一个相同的输入 x ，如图 8-22 所示。

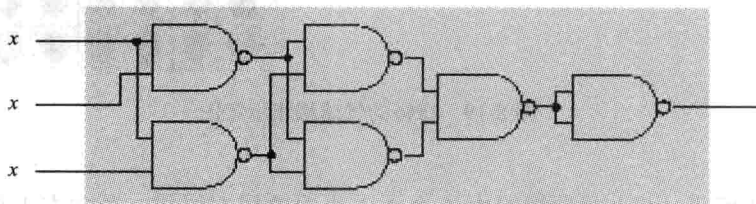


图 8-22 与非门输入电路

请把其中一些输入设置为常数,用最少的 x 完成相同功能。输出任意方案即可。如图 8-23 所示是一个只用一个 x 输入但是可以得到同样结果的电路。

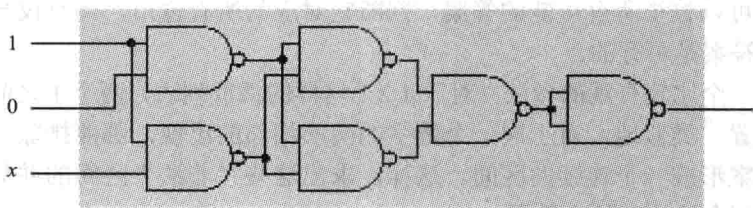


图 8-23 只用一个 x 输入

【分析】

因为只有一个输入 x , 所以整个电路的功能不外乎 4 种: 常数 0、常数 1、 x 及非 x 。先把 x 设为 0, 再把 x 设为 1, 如果二者的输出相同, 整个电路肯定是常数, 任意输出一种方案即可。

如果 $x=0$ 和 $x=1$ 的输出不同, 说明电路的功能是 x 或者非 x , 解至少等于 1。不妨设 $x=0$ 时输出 0, $x=1$ 时输出 1。现在把第一个输入改成 1, 其他仍设为 0 (记这样的输入为 1000...0), 如果输出是 1, 则得到了一个解 $x000\cdots0$ 。

如果 1000...0 的输出也是 0, 再把输入改成 1100...0, 如果输出是 1, 则又得到了一个解 1x00...0。如果输出还是 0, 再尝试 1110...0, 如此等等。由于输入全 1 时输出为 1, 这个算法一定会成功。

问题在于 m 太大, 而每次“给定输入计算输出”都需要 $O(m)$ 时间, 逐个尝试会很慢。好在已经学习了二分查找: 只需二分 1 的个数, 即可在 $O(\log m)$ 次计算之内得到结果, 总时间复杂度为 $O(m \log m)$ 。

例题 8-15 Shuffle 的播放记录 (Shuffle, ACM/ICPC NWERC 2008, UVa 12174)

你正在使用的音乐播放器有一个所谓的乱序功能, 即随机打乱歌曲的播放顺序。假设一共有 s 首歌, 则一开始会给这 s 首歌随机排序, 全部播放完后再重新随机排序、继续播放, 依此类推。注意, 当 s 首歌播放完毕之前不会重新排序。这样, 播放记录里的每 s 首歌都是 $1 \sim s$ 的一个排列。

给出一个长度为 n ($1 \leq s, n \leq 100000$) 的播放记录 (不一定是从最开始记录的) x_i ($1 \leq x_i \leq s$), 你的任务是统计下次随机排序所发生的时间有多少种可能性。

例如, $s=4$, 播放记录是 3, 4, 4, 1, 3, 2, 1, 2, 3, 4, 不难发现只有一种可能性: 前两首是一个段的最后两首歌, 后面是两个完整的段, 因此答案是 1; 当 $s=3$ 时, 播放记录 1, 2, 1 有两种可能: 第一首是一个段, 后两首是另一段; 前两首是一段, 最后一首是另一段。答案为 2。

【分析】

“连续的 s 个数”让你联想到了什么? 没错, 滑动窗口! 这次的窗口大小是“基本”固定的 (因为还需要考虑不完整的段), 因此只需要一个指针; 而且所有数都是 $1 \sim s$ 的整数, 也不需要 STL 的 set, 只需要一个数组即可保存每个数在窗口中出现的次数。再用一个

变量记录在窗口中恰好出现一次的数的个数, 则可以在 $O(n)$ 时间内判断出每个窗口是否满足要求 (每个整数最多出现一次)。

这样, 就可以枚举所有可能的答案, 判断它对应的所有窗口, 当且仅当所有窗口均满足要求时这个答案是可行的。

本题还有一个比较直观的做法: 对于 1 2 1 这样的播放列表, 两个 1 之间必然存在一个窗口的交界位置。类似地, 对于同一个数字的两次相邻的出现, 都能排除一些答案, 而且排除的那些答案形成一个连续的区间。这样, 求出这些“非法”区间的并集, 然后求出总长度, 就能得到合法答案的个数了。

例题 8-16 不无聊的序列 (Non-boring sequences, CERC 2012, UVa1608)

如果一个序列的任意连续子序列中至少有一个只出现一次的元素, 则称这个序列是不无聊 (non-boring) 的。输入一个 n ($n \leq 200000$) 个元素的序列 A (各个元素均为 10^9 以内的非负整数), 判断它是不是不无聊的。

【分析】

不难想到整体思路: 在整个序列中找一个只出现一次的元素, 如果不存在, 则这个序列不是不无聊的; 如果找到一个只出现一次的元素 $A[p]$, 则只需检查 $A[1 \cdots p-1]$ ^① 和 $A[p+1 \cdots n]$ 是否满足条件 (想一想, 为什么)。设长度为 n 的序列需要 $T(n)$ 时间, 则有 $T(n) = \max\{T(k-1) + T(n-k) + \text{找到唯一元素 } k \text{ 的时间}\}$ 。这里取 $\max$ 是因为要看最坏情况。

如何找唯一元素? 如果事先算出每个元素左边和右边最近的相同元素 (还记得《唯一的雪花》吗?), 则可以在 $O(1)$ 时间内判断在任意一个连续子序列中, 某个元素是否唯一。如果从左边找, 最坏情况下唯一元素是最后一个元素, 因此

$$T(n) = T(n-1) + O(n) \geq T(n) = O(n^2)$$

从右往左找也一样, 只不过最坏情况变成了“唯一元素是第一个元素”, 但时间复杂度不变。那么, 从两边往中间找会怎样? 此时 $T(n) = \max\{T(k) + T(n-k) + \min(k, n-k)\}$, 刚才的最坏情况 (即第一个元素或最后一个元素是唯一元素) 变成了 $T(n) = T(n-1) + O(1)$ (因为一下子就找到唯一元素了), 即 $T(n) = O(n)$ 。而此时的最坏情况是唯一元素在中间的情况, 它满足经典递推式 $T(n) = 2T(n/2) + O(n)$, 即 $T(n) = O(n \log n)$ 。

例题 8-17 不公平竞赛 (Foul Play, ACM/ICPC NWERC 2012, UVa1609)

n 支队伍 ($2 \leq n \leq 1024$, 且 n 是 2 的整数幂) 打淘汰赛, 每轮都是两两配对, 胜者进入下一轮, 如图 8-24 所示。

每支队伍的实力固定, 并且已知每两支队伍之间的一场比赛结果 (“实力固定”是指, 例如, 队伍 1 曾经胜过队伍 2, 则二者在今后的交锋中队伍 1 总会获胜)。你喜欢 1 号队。虽然它不一定是最强的, 但是它可以直接打败其他队伍中的至少一半, 并且对于每支 1 号队不能直接打败的队伍 t , 总是存在一支 1 号队能直接打败的队伍 t' 使得 t' 能直接打败 t 。问: 是否存在一种比赛安排, 使得 1 号队夺冠?

【分析】

首先从简单情况分析。 $n=2$ 时, 只有 1 号队伍和另外一支队伍。1 号队伍肯定能打败对

^① $A[1 \cdots p-1]$ 表示子序列 $A[1], A[2], \dots, A[p-1]$ 。

手，因为1号队伍能打败至少一半的队伍，此时“一半的队伍”就是这个唯一的对手。

注意到 n 是 2 的整数幂，所以每次都会恰好淘汰一半的队伍。如果能设计一轮赛程，使得比赛之后所有队伍的情况仍然满足题目的两个条件，则 $\log_2 n$ 次之后 1 号队伍夺冠。由于这两个条件非常重要，下面给它们编号。

条件 1：1 号队能直接打败一半的队伍。

条件 2：对于不能直接打败的队伍 t ，存在队伍 t' 使得 1 号队能打败 t' ，且 t' 能打败 t 。

用黑色代表强队（即 1 号队不能直接打败的队伍），再用灰色代表“有用的队”，即能打败某个黑色队但不能打败 1 号队的队伍（说它们有用是因为可以间接打败黑色队），最后用问号代表 1 号队能打败的队伍（可能是灰色也可能不是，但一定不是黑色）。将赛程安排分为 4 个阶段，如图 8-25 所示。

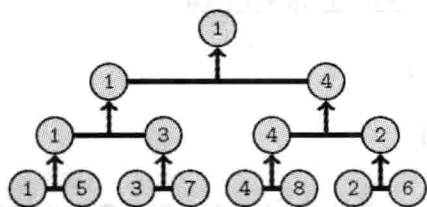


图 8-24 不公平竞赛示意图

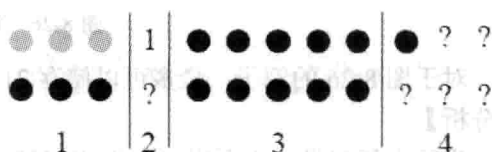


图 8-25 赛程安排的 4 个阶段

阶段 1：首先需要尽量“消灭”黑色队，即依次考虑每一个黑色队，选一个能打败且还没安排对手（称为“配对”）的灰色队。这个阶段结束后，灰色队和黑色队都可能有一些没配对，但有一点是肯定的：已配对的灰色队足以打败现在的所有黑色队。也就是说，对于任意黑色队（不管有没有配对），都至少会输给一支已配对的灰色队。

阶段 2：接下来给 1 号队任选一个能打败的。这个选择一定可以成功，否则说明 1 号队能打败的队伍不到一半，和假设矛盾。

阶段 3：把剩下的黑色队伍任意配对，任它们“自相残杀”，不管谁赢都无所谓。注意，如果前两个阶段结束后没有配对的黑色队伍有奇数个，阶段 3 之后会有一支黑色队留到第 4 阶段。

阶段 4：剩下的队伍（可能需要加上阶段 3 后剩下的一支黑色队）任意配对。

下面看这一轮结束后，题目中的各个条件是否依然满足。

条件 1：粗略地说，阶段 1 中的黑色队全军覆没，且阶段 3 中会消灭一半黑色队，所以总共至少消灭了一半的黑色队。一轮比赛之后，队伍总数减半，而黑色队数目也减半，因此条件 1 仍满足。细心的读者可能会说：如果阶段 4 中有一支黑色队，而阶段 1 完全不存在，则消灭的黑色队不到一半。幸运的是，这样的情况并不存在，因为根据条件 2，灰色队伍至少有一支（但有可能只有一支——即这只强大的灰色队可以消灭所有黑色队）。

条件 2：此条件之前已经证明过了，阶段 1 中灰色队伍联合起来可以打败所有黑色队伍，而这些灰色队伍全都晋级到下一轮。

这样就成功解决了本题。

例题 8-18 洞穴 (Cave, ACM/ICPC CERC 2009, UVa1442)

一个洞穴的宽度为 n ($n \leq 10^6$) 个片段组成。已知位置 $[i, i+1]$ 处的地面高度 p_i 和顶的高

度 s_i ($0 \leq p_i < s_i \leq 1000$), 要求在这个洞穴里储存尽量多的燃料, 使得在任何位置燃料都不会碰到顶 (但是可以无限接近), 如图 8-26 所示。

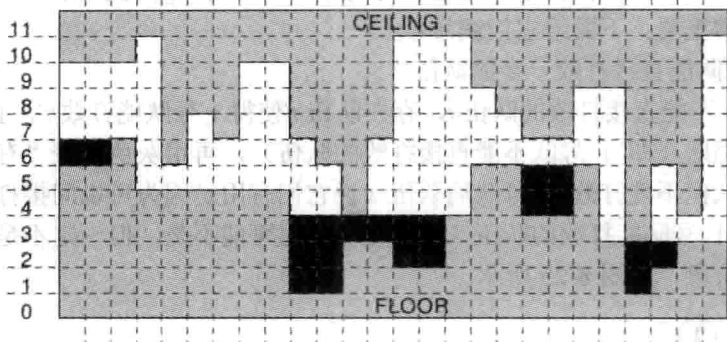


图 8-26 洞穴问题示意图

对于图 8-26 的例子, 最多可以储存 21 单位的燃料。

【分析】

为了方便起见, 下面用“水”来代替题目中的燃料。根据物理定律, 每一段有水的连续区间, 水位高度必须相等, 且水位必须小于等于区间内的最低天花板高度, 因此位置 $[i, i+1]$ 处的水位满足 $h \leq s_i$, 且从 (i, h) 出发往左右延伸出的两条射线均不会碰到天花板 (即两条射线将一直延伸到洞穴之外或先碰到地板之间的“墙壁”) 的最大 h 。如果这样的 h 不存在, 则规定 $h = p_i$ (也就是“没水”)。

这样, 可以先求出“往左延伸不会碰到天花板”的最大值 $h_1(i)$, 再求“往右延伸不会碰到天花板”的最大值 $h_2(i)$, 则 $h_i = \min\{h_1(i), h_2(i)\}$ 。根据对称性, 只考虑 $h_1(i)$ 的计算。

从左到右扫描。初始时设水位 $\text{level} = s_0$, 然后依次判断各个位置 $[i, i+1]$ 处的高度。

- 如果 $p[i] > \text{level}$, 说明水被“隔断”了, 需要把 level 提升到 p_i 。
- 如果 $s[i] < \text{level}$, 说明水位太高, 碰到了天花板, 需要把 level 下降到 s_i 。
- 位置 $[i, i+1]$ 处的水位就是扫描到位置 i 时的 level 。

不难发现, 两次扫描的时间复杂度均为 $O(n)$, 总时间复杂度为 $O(n)$ 。

例题 8-19 贩卖土地 (Selling Land, ACM/ICPC NWERC 2010, UVa 12265)

输入一个 $n \times m$ ($1 \leq n, m \leq 1000$) 矩阵, 每个格子可能是空地, 也可能是沼泽。对于每个空地格子, 求出以它为右下角的空矩形的最大周长, 然后统计每个周长出现了多少次。图 8-27 中标注了 3 个位置的最大空矩形, 其周长分别是 6, 10, 12。如果统计完所有 20 个空地, 答案是 6×4 (表示周长为 4 的矩形有 6 个)、 5×6 、 5×8 、 3×10 、 1×12 。

【分析】

按照从上到下的顺序处理每一行, 在每一行中从左到右处理每个格子 (以下称为“当前格”), 找出以该格子为右下角的最大周长矩形 (以下简称最优矩形)。只要找到了以每个格子为右下角的最优矩形, 本题就可以得到解决。

如图 8-28 所示, 当前行是图的最下行, 当前列是图的最右列 (后同)。假定“当前格”已经固定, 则只需要再确定一个左上角, 就可以得到一个矩形。例如, 把格子 A 作为左上

角, 会得到一个矩形 (以下简称矩形 A), 用粗线标出。黑色长条表示题目中的沼泽, 它们上面的格子不影响答案, 因此没有画出。阴影格子表示该区域无法和当前格构成矩形 (更无法构成最优矩形), 因此可以等同于沼泽处理。换句话说, 可以用数组 $height$ 来描述图 8-28 中的图形, 其中 $height[i]$ 表示第 i 列的空地高度。每次“当前行”往下移时, 可以用 $O(m)$ 时间更新 $height$ 数组。

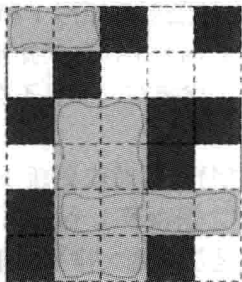


图 8-27 3 个位置的最大空矩形

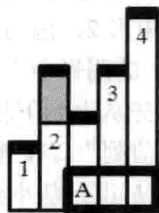
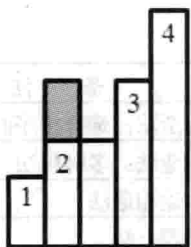


图 8-28 当前行与当前列

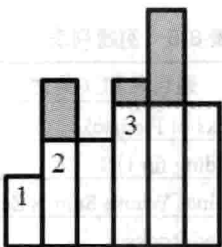
下面考虑图 8-28 中的最优矩形。最优矩形有可能是矩形 A 吗? 不可能, 因为矩形 2 肯定比矩形 A 优 (想一想, 为什么)。矩形 1、2、3、4 哪个最大呢? 在不标明尺寸的情况下无法知道, 需要算一算。不难发现, 在不标明尺寸的情况下, 最优矩形只可能是矩形 1、2、3、4 四者之一。

现在假定“当前行”固定, 而“当前列”往右移动 (最左列编号为 1)。如图 8-29 所示, 最优矩形左上角可能的位置会发生变化。

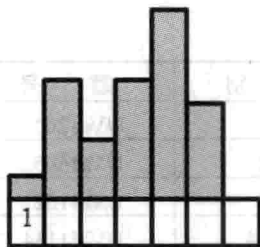
在图 8-29 (a) 中, 最优矩形有 4 种可能, 用 1~4 标记。当前列往右移动一列时, 矩形 4 消失了, 而矩形 3 的高度也变小了 (如图 8-29 (b) 所示)。而当前列再移动一格时, 矩形 2 和矩形 3 都消失了, 矩形 1 也变矮了 (如图 8-29 (c) 所示)。



(a)



(b)



(c)

图 8-29 移动当前列

这就提示要保留最优矩形左上角可能出现的所有位置, 每个位置记为 (c, h) , 表示最左列为 c , 高度为 h 。不难发现, 当 c 从小到大排列时, h 也是从小到大排列的。是不是似曾相识?

没错, 这个“双重有序”的结构和例题“防线”是完全一样的, 不过其他部分有些差别。在例题“防线”中, 需要用二分查找来找到想要的元素, 不过在本题中, 不是要找一个元素, 而是要找所有元素的最大值。这里的“最大”是指以 (c, h) 为左上角、当前格子为右下角的矩形周长最大。格子 (c, h) 所对应的矩形周长为 $2(c_0 - c + 1 + h)$, 其中 c_0 是当前列。不难

发现，周长最大意味着 $h-c$ 最大，与 c_0 无关。

既然与 c_0 无关，那么任意两个矩形的大小关系永远都不会改变。这岂不是说明只需要保存一个让 $h-c$ 最大的 (c, h) ？并非如此。在图 8-29 (a) 中，最优矩形可能是矩形 4，但“当前列”右移一格后，矩形 4 消失了！如果没有保存矩形 1, 2, 3，一旦矩形 4 消失，就什么也求不出了。类似地，如果图 8-29 (a) 中的最优矩形是矩形 3，虽然“当前列”右移之后没有消失，但却变矮了，可能不再是最优矩形了。这时还是要靠矩形 1 和矩形 2。

但也不是所有矩形都得保存下来。例如，在图 8-29 (a) 中，如果矩形 1 的 $h-c$ 比矩形 2 大，则不用保存矩形 2，因为只要矩形 2 还在，矩形 1 肯定在，而且不会变矮。所以矩形 2 “永远活在矩形 1 的阴影中”，不可能成为最优矩形。

总结一下。首先从上到下枚举“当前行”，在处理每一行时先更新 `height` 数组，然后从左到右枚举“当前列”。在移动“当前列”的过程中，保存若干个 (c, h) ，按照 c 从小到大排列成有序表，则 h 也是从小到大排列，并且 $h-c$ 也是从小到大排列。根据上述分析，可以在 $O(1)$ 时间内求出每个当前格对应的最优矩形（因为最后一个矩形就是最优的），然后根据需求从右到左删除一些矩形（也可能不删除），并且可能会把最右边的矩形变矮。然后，当且仅当新矩形的 $h-c$ 比它左边的矩形大时，加到表的最右边。由于添加和删除都在表的最右端，用一个栈来实现即可。

8.7 训练参考

本章是竞赛篇中的第一章节，例题难度和前 7 章相比有较大幅度的提升。如果希望在高水平算法竞赛中取得好成绩，本章中的所有例题（见表 8-5）都是必须掌握的。另一方面，在初学阶段，不必强求掌握表 8-5 中带星号的例题，只需要尽量掌握未带星号的例题。

表 8-5 例题列表

类 别	题 号	题目名称（英文）	备 注
例题 8-1	UVa120	Stacks of Flapjacks	构造法；选择排序的思想
例题 8-2	UVa1605	Building for UN	构造法；多种解法
例题 8-3	UVa1152	4 Values Whose Sum is Zero	中途相遇法
*例题 8-4	UVa11134	Fabled Rooks	问题分解
例题 8-5	UVa11054	Wine trading in Gergovia	等价转换
*例题 8-6	UVa1606	Amphiphilic Carbon Molecules	极角扫描法
例题 8-7	UVa11572	Unique snowflakes	滑动窗口
**例题 8-8	UVa1471	Defense Lines	使用数据结构加速算法
**例题 8-9	UVa1451	Average	数形结合
例题 8-10	UVa714	Copying Books	二分法
例题 8-11	UVa10954	Add All	Huffman 编码
例题 8-12	UVa12627	Erratic Expansion	递归
例题 8-13	UVa11093	Just Finish it up	模拟法

续表

类 别	题 号	题目名称 (英文)	备 注
*例题 8-14	UVa1607	Gates	二分法
例题 8-15	UVa12174	Shuffle	滑动窗口或问题转换
*例题 8-16	UVa1608	Non-boring sequences	分治法; 中途相遇法的思路
*例题 8-17	UVa1609	Foul Play	递归; 构造法
*例题 8-18	UVa1442	Cave	扫描法
**例题 8-19	UVa12265	Selling Land	扫描法; 状态组织; 单调栈

算法设计方法和技巧五花八门, 因此本章的习题也比前 7 章更多。建议读者阅读所有题目, 选择自己有思路的题目深入思考并编程实现。排列在前面的习题总体上会更简单一些, 但也有些例外。这些习题的整体难度比前 7 章大, 读者需要做好花费更多时间的心理准备。

习题 8-1 装箱 (Bin Packing, SWERC 2005, UVa1149)

给定 N ($N \leq 10^5$) 个物品的重量 L_i , 背包的容量 M , 同时要求每个背包最多装两个物品。求至少要多少个背包才能装下所有的物品。

习题 8-2 聚会游戏 (Party Games, Mid-Atlantic 2012, UVa1610)

输入一个 n ($2 \leq n \leq 1000$, n 是偶数) 个字符串的集合 D , 找一个长度最短的字符串 (不一定在 D 中出现) S , 使得 D 中恰好一半串小于等于 S , 另一半串大于 S 。如果有多解, 输出字典序最小的解。例如, 对于 {JOSEPHINE, JERRY}, 输出 JF; 对于 {FRED, FREDDIE}, 输出 FRED。提示: 本题看似简单, 实际上暗藏陷阱, 需要考虑细致、周全。

本题容易想复杂, 或者把细节想错, 强烈建议读者编程实现。

习题 8-3 比特变换器 (Bits Equalizer, SWERC 2012, UVa12545)

输入两个等长 (长度不超过 100) 的串 S 和 T , 其中 S 包含字符 0, 1, ?, 但 T 只包含 0 和 1。你的任务是用尽量少的步数把 S 变成 T 。每步有 3 种操作: 把 S 中的 0 变成 1; 把 S 中的 “?” 变成 0 或者 1; 交换 S 中任意两个字符。例如, 01??00 经过 3 步可以变成 001010 (方法是先把两个问号变成 1 和 0, 再交换两个字符)。

习题 8-4 奖品的价值 (Erasing and Winning, UVa11491)

你是一个电视节目的获奖嘉宾。主持人在黑板上写出一个 n 位整数 (不以 0 开头), 邀请你删除其中的 d 个数字, 剩下的整数便是你所得到的奖品的价值。当然, 你希望这个奖品价值尽量大。 $1 \leq d < n \leq 10^5$ 。

习题 8-5 折纸痕 (Paper Folding, UVa177)

你喜欢折纸吗? 给你一张很大的纸, 对折以后再对折, 再对折……每次对折都是从右往左折, 因此在折了很多次以后, 原先的大纸会变成一个窄窄的纸条。现在把这个纸条沿着折纸的痕迹打开, 每次都只打开 “一半”, 即把每个痕迹做成一个直角, 那么从纸的一端沿着和纸面平行的方向看过去, 会看到一个美妙的曲线。

例如, 如果对折了 4 次, 那么打开以后将看到如图 8-30 所示的曲线。注意, 该曲线是不自交的, 虽然有两个转折点重合。给出对折的次数, 请编程绘出打开后生成的曲线。

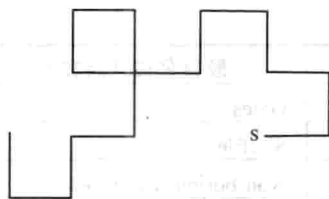


图 8-30 直角折痕

习题 8-6 起重机 (Crane, ACM/ICPC CERC 2013, UVa1611)

输入一个 $1 \sim n$ ($1 \leq n \leq 10000$) 的排列, 用不超过 9^6 次操作把它变成升序。每次操作都可以选一个长度为偶数的连续区间, 交换前一半和后一半。例如, 输入 5, 4, 6, 3, 2, 1, 可以执行 1, 2 先变成 4, 5, 6, 3, 2, 1, 然后执行 4, 5 变成 4, 5, 6, 2, 3, 1, 然后执行 5, 6 变成 4, 5, 6, 2, 1, 3, 然后执行 4, 5 变成 4, 5, 6, 1, 2, 3, 最后执行操作 1, 6 即可。

提示: $2n$ 次操作就足够了。

习题 8-7 生成排列 (Generating Permutations, UVa11925)

输入一个 $1 \sim n$ ($1 \leq n \leq 300$) 的排列, 用不超过 $2n^2$ 次操作把它变成升序。操作只有两种: 交换前两个元素 (操作 1); 把第一个元素移动到最后一位 (操作 2)。

例如, 输入排列为 4, 2, 3, 1, 一个合法操作序列为 12122, 具体步骤是: 4231 $\rightarrow$ 2431 $\rightarrow$ 4312 $\rightarrow$ 3412 $\rightarrow$ 4123 $\rightarrow$ 1234。

习题 8-8 猜名次 (Guess, ACM/ICPC Beijing 2006, UVa1612)

有 n ($n \leq 16384$) 位选手参加编程比赛。比赛有 3 道题目, 每个选手的每道题目都有一个评测之前的预得分 (这个分数和选手提交程序的时间相关, 提交得越早, 预得分越大)。接下来是系统测试。如果某道题目未通过测试, 则该题的实际得分为 0 分, 否则得分等于预得分。得分相同的选手, ID 小的排在前面。

问是否能给出所有 $3n$ 个得分以及最后的实际名次。如果可能, 输出最后一名的最高可能得分。每个预得分均为小于 1000 的非负整数, 最多保留两位小数。

习题 8-9 K 度图的着色 (K-Graph Oddity, ACM/ICPC NEERC 2010, UVa1613)

输入一个 n ($3 \leq n \leq 9999$) 个点 m 条边 ($2 \leq m \leq 100000$) 的连通图, n 保证为奇数。设 k 为最小的奇数, 使得每个点的度数不超过 k , 你的任务是给图中的结点涂上颜色 $1 \sim k$, 使得相邻结点的颜色不同。多解时输出任意解。输入保证有解。如图 8-31 所示, $k=3$ 。

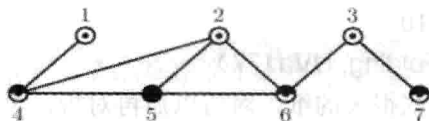


图 8-31 连通图

习题 8-10 奇怪的股市 (Hell on the Markets, ACM/ICPC NEERC 2008, UVa1614)

输入一个长度为 n ($n \leq 100000$) 的序列 a , 满足 $1 \leq a_i \leq i$, 要求确定每个数的正负号, 使得所有数的总和为 0。例如 $a = \{1, 2, 3, 4\}$, 则设 4 个数的符号分别是 1, -1, -1, 1 即可 ($1 - 2 - 3 + 4 = 0$), 但如果 $a = \{1, 2, 3, 3\}$, 则无解 (输出 No)。

习题 8-11 高速公路 (Highway, ACM/ICPC SEERC 2005, UVa1615)

给定平面上 n ($n \leq 10^5$) 个点和一个值 D , 要求在 x 轴上选出尽量少的点, 使得对于给定的每个点, 都有一个选出的点离它的欧几里德距离不超过 D 。

习题 8-12 顾客是上帝 (Keep the Customer Satisfied, ACM/ICPC SWERC 2005, UVa1153)

有 n ($n \leq 800000$) 个工作, 已知每个工作需要的时间 q_i 和截止时间 d_i (必须在此之前完成), 最多能完成多少个工作? 工作只能串行完成。第一项任务开始的时间不早于时刻 0。

习题 8-13 外星人聚会 (Meeting with Aliens, UVa10570)

输入 $1 \sim n$ 的一个排列 ($3 \leq n \leq 500$), 每次可以交换两个整数。用最少的交换次数把排列变成 $1 \sim n$ 的一个环状排列。

习题 8-14 商队抢劫者 (Caravan Robbers, ACM/ICPC NEERC 2012, UVa1616)

输入 n 条线段, 把每条线段变成原线段的一条子线段, 使得改变之后所有线段等长且不相交 (但是端点可以重合)。输出最大长度 (用分数表示)。例如, 有 3 条线段 $[2, 6]$, $[1, 4]$, $[8, 12]$, 则最优方案是分别变成 $[3.5, 6]$, $[1, 3.5]$, $[8, 10.5]$, 输出 $5/2$ 。

习题 8-15 笔记本 (Laptop, ACM/ICPC Daejeon 2012, UVa1617)

有 n ($1 \leq n \leq 100000$) 条长度为 1 的线段, 确定它们的起点 (必须是整数), 使得第 i 条线段在 $[r_i, d_i]$ 之间 ($0 \leq r_i \leq d_i \leq 1000000$)。输入保证 $r_i \leq r_j$, 当且仅当 $d_i \leq d_j$, 且保证有解。输出“空隙”数目的最小值。如图 8-29 所示, 5 条线段的范围分别为 $[4, 8]$, $[1, 3]$, $[8, 10]$, $[0, 3]$, $[6, 8]$, 一组解如图 8-32 所示, 空隙有 3 个。

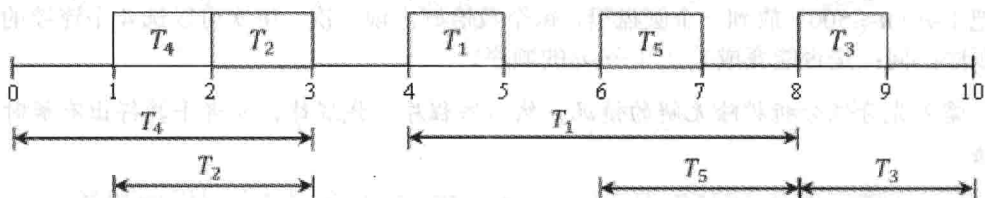


图 8-32 5 条线段范围

最优解如图 8-33 所示, 空隙数目仅为 1 (T_2 和 T_5 之间)。

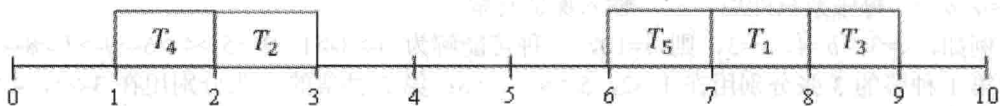


图 8-33 最优解

习题 8-16 弱键 (Weak Key, ACM/ICPC Seoul 2004, UVa1618)

给出 k ($4 \leq k \leq 5000$) 个互不相同的整数组成的序列 N_i , 判断是否存在 4 个整数 N_p 、 N_q 、 N_r 和 N_s ($1 \leq p < q < r < s \leq k$), 使得 $N_q > N_s > N_p > N_r$ 或者 $N_q < N_s < N_p < N_r$ 。

习题 8-17 最短子序列 (Smallest Sub-Array, UVa11536)

有 n ($n \leq 10^6$) 个 $0 \sim m-1$ ($m \leq 1000$) 的整数组成一个序列。输入 k ($k \leq 100$), 你的任务是找一个尽量短连续子序列 $(x_a, x_{a+1}, x_{a+2}, \dots, x_{b-1}, x_b)$, 使得该子序列包含 $1 \sim k$ 的所有整数。

例如, $n=20$, $m=12$, $k=4$, 序列为 1 (2 3 7 1 12 9 11 9 6 3 7 5 4) 5 3 1 10 3 3, 括号内部

分是最优解。如果不存在满足条件的连续子序列, 输出 sequence nai。

习题 8-18 感觉不错 (Feel Good, ACM/ICPC NEERC 2005, UVa1619)

给出一个长度为 n ($n \leq 100000$) 的正整数序列 a_i , 求出一段连续子序列 $a_l, \dots, a_r$, 使得 $(a_l + \dots + a_r) * \min\{a_l, \dots, a_r\}$ 尽量大。

习题 8-19 球场 (Cricket Field, ACM/ICPC NEERC 2002, UVa 1312)

一个 $W * H$ ($1 \leq W, H \leq 10000$) 网格里有 n ($0 \leq n \leq 100$) 棵树, 如图 8-34 所示, 要求找一个最大空正方形。

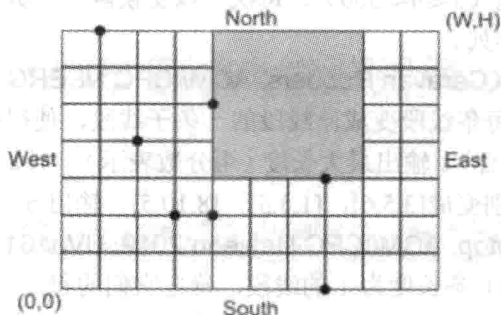


图 8-34 球场

习题 8-20 懒惰的苏珊 (Lazy Susan, ACM/ICPC Danang 2007, UVa1620)

把 $1 \sim n$ ($n \leq 500$) 放到一个圆盘里, 每个数恰好出现一次。每次可以选 4 个连续的数字翻转顺序。问: 是否能变成 $1, 2, 3, \dots, n$ 的顺序?

提示: 需要先奇偶分析排除无解的情况, 然后写程序、找规律, 或者手算得出有解时的构造算法。

习题 8-21 跳来跳去 (Jumping Around, ACM/ICPC NEERC 2012, UVa1621)

你的任务是数轴上的 0 点出发, 访问 $0, 1, 2, \dots, n$ 各一次, 在任意点终止。需要用票才能从一个点到达另一个点。有 3 种票, 跳跃长度为 1, 2, 3, 分别有 a, b, c 张 ($3 \leq a, b, c \leq 5000$), 且 $n = a + b + c$ 。每张票只能用一次。输入保证有解。

例如, $a=3, b=4, c=3$, 则 $n=10$, 一种可能解为 $0 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 9 \rightarrow 7 \rightarrow 8 \rightarrow 10$, 其中第 1 种票的 3 张分别用在 $1 \rightarrow 2, 5 \rightarrow 4, 7 \rightarrow 8$; 第 2 种票的 4 张分别用在 $3 \rightarrow 1, 4 \rightarrow 6, 9 \rightarrow 7, 8 \rightarrow 10$; 第 3 种票的 3 张分别用在 $0 \rightarrow 3, 2 \rightarrow 5, 6 \rightarrow 9$ 。

习题 8-22 机器人 (Robot, ACM/ICPC Beijing 2006, UVa1622)

有一个 $n * m$ ($1 \leq n, m \leq 10^5$) 的网格, 每个格子里都有一个机器人。每次可以发出如下 4 种指令之一: NORTH、SOUTH、EAST、WEST, 作用是让所有机器人往相应方向走一格。如果一个机器人在执行某一命令后走出了网格, 则它会立即炸毁。

给出 4 种指令的总条数 ($0 \leq C_N, C_S, C_W, C_E \leq 10^5$), 求一种指令顺序使得所有机器人执行的命令条数之和最大。炸毁的机器人不再执行命令。

习题 8-23 神龙喝水 (Enter the Dragon, ACM/ICPC CERC 2010, UVa1623)

某城市里有 n 个湖, 每个湖都装满了水。天气预报显示不久的将来会有暴雨。具体来

说,在接下来的 m 天内,每天要么不下雨,要么恰好往一个湖里下暴雨。如果这个湖里已经装满了水,将会引发水灾。为了避免水灾,市长请来一只神龙,可以在每个不下雨的天里喝干一个湖里的水(也可以不喝)。如果以后再往这个干枯的湖里下暴雨,湖会重新被填满,但不会引发水灾。神龙应当如何喝水才能避免水灾? $n \leq 10^6$, $m \leq 10^6$ 。

提示:需要优化算法的时间复杂度。

习题 8-24 龙头滴水 (Faucet Flow, UVa10366)

$x=0$ 的正上方有一个水龙头,以每秒 1 单位体积的速度往下滴水。 $x=-1, -3, \dots, \text{leftx}$ 和 $x=1, 3, 5, \dots, \text{rightx}$ 处各有一个挡板,高度已知。求经过多长时间以后水会流出最左边的挡板或者最右边的挡板。如图 8-35 所示, $\text{leftx}=-3$, $\text{rightx}=3$, 4 个挡板高度分别为 4, 3, 2, 1, 则 6 秒钟之后水会从最右边的挡板溢出。

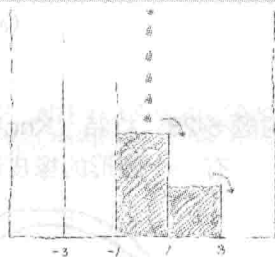


图 8-35 龙头滴水示意图

输入第一行为两个奇数 leftx , rightx ($\text{leftx} \leq -1$, $\text{rightx} \geq 1$), 接下来的各个正整数表示从左到右各个挡板的高度。挡板个数不超过 1000。

习题 8-25 有向图 D 和 E (From D to E and back, UVa11175)

给一个 n 个结点的有向图 D, 可以构造一个图 E: D 的每条边对应 E 的一个结点(例如, 若 D 有一条边 uv , 则 E 有个结点的名字叫 uv), 对于 D 的两条边 uv 和 vw , E 中的两个结点 uv 和 vw 之间连一条有向边。E 中不包含其他边。

输入一个 m 个结点 k 条边的图 E ($0 \leq m \leq 300$), 判断是否存在对应的图 D。E 中各个结点的编号为 $0 \sim m-1$ 。

提示: 虽然题目中 $m \leq 300$, 实际上可以解决的规模远超过这个限制的问题。

习题 8-26 找黑圆 (Finding [B]lack Circles, Rujia Liu's Present 6, UVa12559)

输入一个 $h \times w$ 的黑白图像 ($30 \leq w, h \leq 100$), 你的任务是找出图像中的圆。每个像素都是 1×1 的正方形, 左上角像素的中心坐标为 $(0,0)$, 右下角像素的中心坐标为 $(w-1, h-1)$ 。对于一个圆, 它的圆周穿过(只是接触到像素边界不算)的像素都会被涂黑(用 1 表示)。没有被任何圆穿过的像素仍然是白色(用 0 表示)。圆心保证在整点处, 半径保证是 $1 \sim 5$ 之间的整数。最多有 2% 的黑点会变成白点。

提示: 方法有多种, 尽情发挥创造力吧。

习题 8-27 海盗的宝箱 (Pirate Chest, ACM/ICPC World Finals 2013, UVa1580)

有一个顶面为 $m \times n$ 的池塘, 已知每个格子 (i,j) 的水深 $d(i,j)$ ($1 \leq i \leq m$, $1 \leq j \leq n$, $0 \leq d(i,j) \leq 10^9$)。要求放一个长和宽分别不超过 a 和 b (但长宽可以交换, 高度任意)、体积尽量大的长方体, 使得长方体的顶面严格位于水平面之下。注意, 池塘里放入长方体后, 水面会上升(即使长方体紧紧贴住墙壁)。池塘四周是足够高的墙壁。

如图 8-36 (b) 中放了一个底面为 1×3 , 高度为 1 的长方体, 体积为 3; 图 8-36 (c) 中放了一个 $1 \times 2 \times 2$ 的长方体, 体积为 4。输入保证 $a \times b$ 不足以覆盖整个池塘。 $1 \leq a, b, m, n \leq 500$ 。

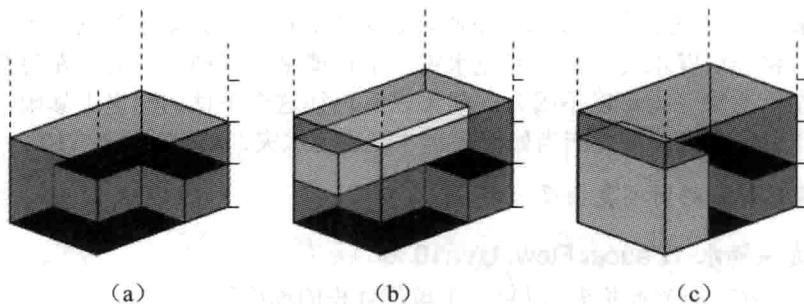


图 8-36 水池示意图

习题 8-28 打结 (Knots, ACM/ICPC ACM/ICPC Jakarta 2012, UVa1624)

有一个圆形的橡皮圈，可以对它进行 Self loop 和 Passing 两种操作，如图 8-37 所示。

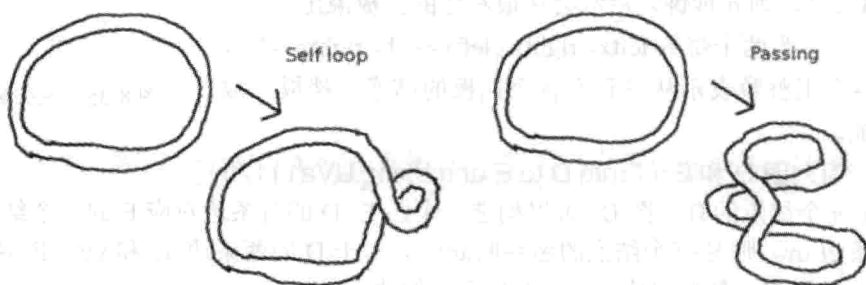


图 8-37 Self loop 和 Passing 操作

输入一个橡皮圈，判断是否可以由原始的圆形橡皮圈经过重复的两种操作得到。橡皮圈的描述方法如下：首先是两个正整数 L 和 P ($L \leq 10^6$, $P \leq 5000$)，然后把橡皮圈上的 L 个位置按顺序编号为 $0 \sim L-1$ ，接下来是 P ($1 \leq P \leq 5000$) 个整数对 (A_i, B_i) ，表示从上往下俯视时位置 A_i 挡住位置 B_i ($0 \leq A_i, B_i < L$)。输入保证 $0 \sim L-1$ 中的每个位置最多在一个数对中出现。

如图 8-38 所示，图 8-38 (a) 和图 8-38 (b) 都可以由原始橡皮圈得到，但图 8-38 (c) 不可以。其中图 8-38 (a) 的 $L=20$, $P=5$, 5 个数对分别是 $(0,8)$, $(2,10)$, $(4,12)$, $(15,5)$, $(18,7)$ 。

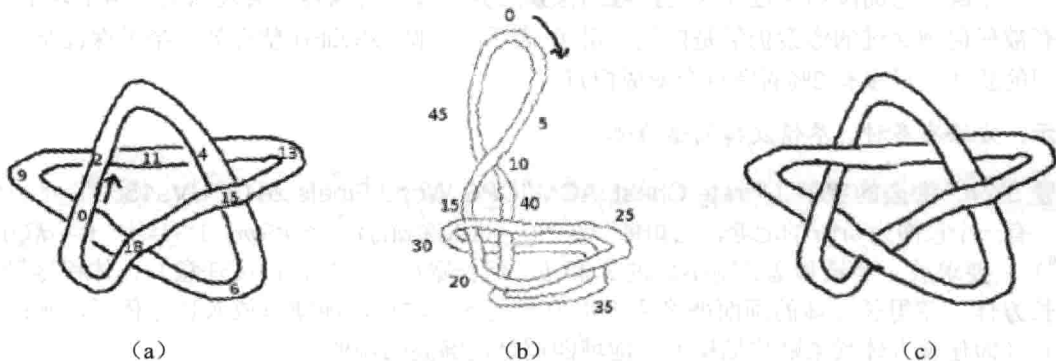


图 8-38 橡皮圈效果

提示： 本题不需要特别的数学知识或算法知识，但需要仔细思考。